

Defensa completa SOS2526-29

Documento canonico del proyecto para la defensa. Es la fuente principal para estudiar, modificar y explicar el proyecto; el `README.md` queda como entrada breve al repositorio.

Sustituye toda la documentacion de defensa que antes estaba repartida en varias guias. Esas guias antiguas ya no se conservan como fuentes independientes dentro de `docs/`.

Objetivo: que una persona que no conoce el repositorio pueda entender que hace el proyecto, como se instala, como se ejecuta, como esta organizado, que rutas existen, que funciones importan, como fluyen los datos y como defenderlo ante un profesor o tribunal.

Nota de nombre: el archivo real del repositorio es `docs/DEFENSA_COMPLETA.md`. Cuando en conversaciones o planes aparezca `DEFENSA_COMPLETA.md`, se refiere a este documento.

Fuente actualizada para estudiar y modificar: este Markdown. Si se quiere una copia PDF para imprimir, debe regenerarse desde esta version.

Indice

- 1. Resumen ejecutivo - pag. 2
- 2. Problema, objetivos y publico - pag. 3
- 3. Equipo y recursos - pag. 3
- 4. Stack tecnologico - pag. 4
- 5. Arquitectura general - pag. 5
- 6. Estructura real de carpetas - pag. 6
- 7. Instalacion, configuracion y ejecucion - pags. 7-8
- 8. Scripts, tests y validaciones - pag. 9
- 9. Rutas utiles para abrir - pag. 10
- 10. Modelo de datos - pag. 11
- 11. Backend: arranque y responsabilidades - pags. 11-12
- 12. APIs REST del proyecto - pags. 13-14
- 13. Frontend: rutas, servicios y pantallas - pag. 15
- 14. Parte LCC: `citys-stats` - pags. 16-18
- 15. Integraciones LCC - pags. 19-21
- 16. Funciones importantes explicadas - pags. 22-48
- 17. Flujos de ejecucion - pags. 49-62
- 18. Codigos HTTP y contrato REST - pags. 63-66
- 19. Cambios que pueden pedir en directo - pags. 67-71
- 20. Errores comunes y soluciones - pags. 72-74

- 21. Decisiones tecnicas - pag. 75
- 22. Limitaciones y mejoras futuras - pag. 76
- 23. Guion de defensa oral - pags. 77-89
- 24. Preguntas dificiles y respuestas - pags. 90-91
- 25. Glosario para personas no tecnicas - pag. 92
- 26. Checklists finales - pag. 93
- 27. Anexos tecnicos - pags. 94-95

Ruta de lectura recomendada para defender LCC `citys-stats` :

1. Primero lee las secciones 1 a 5 para entender el proyecto comun y poder abrir la defensa sin perderte.
2. Despues lee las secciones 10 a 15 para centrarte en el modelo, API, frontend e integraciones de `citys-stats`.
3. Estudia muy bien la seccion 16: ahi esta el codigo de funciones, como explicarlas y como modificarlas.
4. Usa las secciones 17 a 20 para responder flujos, codigos HTTP, cambios en directo y errores.
5. Termina con las secciones 23, 24 y 26 para practicar guion oral, preguntas dificiles y checklist final.

1. Resumen ejecutivo

`SOS2526-29` es una aplicacion web de la asignatura SOS2526. Tiene backend y frontend y permite gestionar, consultar, visualizar e integrar tres recursos de datos:

- `natural-disasters` : desastres naturales por pais y anio.
- `citys-stats` : poblacion estimada de ciudades para 2025.
- `wine-stats` : informacion de vinos, precios, tipos y caracteristicas.

La aplicacion permite:

- Listar datos.
- Buscar y filtrar datos.
- Crear registros.
- Editar registros en pantallas separadas.
- Borrar registros concretos.
- Borrar colecciones completas.
- Cargar datos iniciales.
- Mostrar graficas con Highcharts.
- Mostrar mapas.
- Consumir integraciones externas mediante proxy propio.

Frase corta de defensa:

Nuestro proyecto es una aplicacion web con Node.js, Express, NeDB y Svelte. El backend ofrece una API REST para tres recursos: `natural-disasters`, `citys-stats` y `wine-stats`. El frontend consume esas APIs con `fetch`, muestra CRUDs, graficas, mapas e integraciones. Los datos se guardan en archivos NeDB y las APIs externas se consumen desde el backend para controlar errores y normalizar las respuestas.

2. Problema, objetivos y publico

Problema que resuelve

El proyecto organiza varias fuentes de datos heterogeneas en una aplicacion unica. En vez de consultar JSON crudo o APIs separadas, el usuario puede navegar por pantallas, ver tablas, cargar datos, hacer operaciones CRUD y entender los datos mediante visualizaciones.

Objetivos funcionales

- Exponer una API REST para cada recurso.
- Mantener versiones de API cuando un recurso evoluciona.
- Persistir datos localmente con NeDB.
- Ofrecer una interfaz web usable con Svelte.
- Validar entradas para evitar datos incompletos o inconsistentes.
- Probar el contrato de API con Newman.
- Probar flujos de usuario con Playwright.
- Mostrar visualizaciones individuales y grupales.
- Integrar APIs externas sin que el navegador dependa directamente de ellas.

Objetivos de defensa

- Saber explicar el arranque completo desde `npm start`.
- Saber localizar cada capa: servidor, API, servicio frontend y pantalla.
- Saber justificar REST, versionado, codigos HTTP, validaciones y proxy.
- Saber hacer cambios pequenos en directo sin perderse.
- Saber responder por que `citys-stats` usa v2 para CRUD y v1 para integraciones.

Publico objetivo

- Profesor o evaluador de SOS2526.
- Companeros del grupo.
- Cualquier persona tecnica que quiera ejecutar o revisar el proyecto.
- Cualquier stakeholder no tecnico que quiera entender el resultado final.

3. Equipo y recursos

Miembro	Recurso	API principal	Frontend
Rufino Moreno Pacheco	wine-stats	/api/v1/wine-stats	/wine-stats
Luis Cortes Cobos	citys-stats	/api/v2/citys-stats	/citys-stats
Alberto Lirola Gomez	natural-disasters	/api/v2/natural-disasters	/natural-disasters

Repositorio:

```
https://github.com/gti-sos/SOS2526-29
```

Aplicacion desplegada:

```
https://sos2526-29.onrender.com/
```

4. Stack tecnologico

Tecnologia	Donde aparece	Para que se usa
Node.js	index.js , scripts npm	Ejecutar el backend
Express	index.js , src/back/**	Definir servidor y rutas REST
@seald-io/nedb	index.js , src/back/*.db	Persistencia local en archivos
CORS	index.js	Permitir llamadas desde Vite en desarrollo
Svelte	frontend-group/src	Construir la interfaz
Vite	frontend-group/vite.config.js	Desarrollo y build del frontend
Highcharts	analytics e integraciones	Graficas, mapas y widgets
@highcharts/map-collection	mapa de ciudades	TopoJSON del mapa mundial
Newman	tests/**/*.json	Ejecutar colecciones Postman
Playwright	tests/**/*.e2e	Pruebas end-to-end
start-server-and-test	package.json	Arrancar servidor y lanzar tests

Dependencias principales en package.json :

```
express
@seald-io/nedb
cors
svelte
highcharts
@highcharts/map-collection
leaflet
cool-ascii-faces
```

Dependencias del frontend en frontend-group/package.json :

```
@sveltejs/vite-plugin-svelte
vite
svelte
svelte-spa-router
highcharts
@highcharts/map-collection
```

Nota: aunque aparece `svelte-spa-router`, el enrutado actual no usa rutas `#/`: `App.svelte` descubre pantallas con `import.meta.glob` y resuelve `window.location.pathname`; `navigation.js` usa `history.pushState` para navegar dentro de la SPA.

5. Arquitectura general

El proyecto sigue una arquitectura por capas:

```
Navegador
|
| HTML, CSS, JS compilado
v
Frontend Svelte en public/
|
| fetch HTTP JSON
v
Backend Express en index.js
|
| registra modulos REST
v
src/back/v1 y src/back/v2
|
| consultas y modificaciones
v
Bases NeDB en src/back/*.db

Integraciones externas:

Frontend Svelte -> API propia /api/v1/citys-stats/integrations/... -> fetch desde Express -> APIs
externas JSON
```

Separacion principal:

- `index.js`: arranque, middleware, bases de datos, registro de APIs, frontend estatico y fallback SPA.
- `src/back/v1` y `src/back/v2`: contratos REST por recurso y version.
- `frontend-group/src/services`: funciones `fetch` que encapsulan URLs y errores.
- `frontend-group/src/routes`: pantallas visibles, organizadas por carpetas con `+page.svelte`.
- `public`: build final del frontend que sirve Express en produccion.
- `tests`: validaciones API y navegador.

Frase de defensa:

La aplicación separa responsabilidades: Svelte solo pinta y llama a servicios, Express valida y coordina, NeDB persiste, y las integraciones externas pasan por nuestro backend para evitar problemas de origen y controlar errores.

6. Estructura real de carpetas

Estructura resumida:

```
SOS2526-29/
|-- index.js
|-- package.json
|-- package-lock.json
|-- README.md
|-- docs/
|   |-- DEFENSA_COMPLETA.md
|   |-- DEFENSA_COMPLETA.pdf
|-- src/
|   |-- back/
|       |-- v1/
|           |-- citys-stats.js
|           |-- natural-disasters.js
|           |-- wine-stats.js
|       |-- v2/
|           |-- citys-stats.js
|           |-- natural-disasters.js
|           |-- citys-stats.db
|           |-- natural-disasters.db
|           |-- wine-stats.db
|-- frontend-group/
|   |-- package.json
|   |-- vite.config.js
|   |-- src/
|       |-- App.svelte
|       |-- main.js
|       |-- app.css
|       |-- components/
|           |-- Navbar.svelte
|       |-- lib/
|           |-- navigation.js
|       |-- services/
|           |-- apiBase.js
|           |-- citysStatsApi.js
|           |-- citysStatsIntegrations.js
|           |-- natural-disasters.js
|           |-- wine-stats.js
|       |-- routes/
|           |-- +page.svelte
|           |-- citys-stats/
|           |-- natural-disasters/
|           |-- wine-stats/
|           |-- analytics/
|           |-- integrations/
|-- public/
|-- tests/
|   |-- ALG/
```

```
| |-- LCC/  
| |-- RMP/  
|-- playwright.config.js
```

Archivos clave:

Archivo	Importancia
index.js	Punto de entrada del backend y servidor unico
src/back/v2/citys-stats.js	API principal del CRUD LCC
src/back/v1/citys-stats.js	API v1 LCC, agregados por pais e integraciones
frontend-group/src/App.svelte	Router real de la SPA
frontend-group/src/components/Navbar.svelte	Menu superior
frontend-group/src/routes/+page.svelte	Portada
frontend-group/src/routes/citys-stats/+page.svelte	CRUD LCC
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte	Edicion LCC
frontend-group/src/routes/analytics/+page.svelte	Widget grupal
frontend-group/src/routes/analytics/citys-stats/+page.svelte	Grafico individual LCC
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte	Mapa LCC
frontend-group/src/routes/integrations/citys-stats/+page.svelte	Integraciones LCC
frontend-group/src/services/citysStatsApi.js	Fetch CRUD LCC v2
frontend-group/src/services/citysStatsIntegrations.js	Fetch integraciones LCC v1
frontend-group/vite.config.js	Build hacia ../public

Rutas canonicas LCC:

- La ruta canonica para defensa es `citys-stats`, porque ese es el nombre del recurso publicado.
- Las rutas antiguas `city-stats` se eliminaron para evitar duplicidad en el frontend.

7. Instalacion, configuracion y ejecucion

Requisitos

- Node.js instalado.
- npm instalado.
- Acceso a internet si se van a probar integraciones externas.

Instalacion desde cero

En la raiz del repositorio:

```
npm.cmd install
npm.cmd --prefix frontend-group install
```

Tambien funciona:

```
npm install
npm --prefix frontend-group install
```

En PowerShell se recomienda `npm.cmd` si aparece el error de `npm.ps1`.

Build del frontend

```
npm.cmd run build
```

Que hace:

1. Entra en `frontend-group`.
2. Ejecuta `vite build`.
3. Genera el frontend compilado en `public`.
4. Borra previamente `public` por `emptyOutDir: true`.

Arranque local

```
npm.cmd start
```

Equivale a:

```
node index.js
```

URL local:

```
http://localhost:10000
```

El puerto por defecto esta en `index.js`:

```
const port = process.env.PORT || 10000;
```

En Render manda `process.env.PORT`; en local, si no se define, usa `10000`.

Desarrollo frontend con Vite

```
npm.cmd run dev-front
```

Ese script ejecuta:

```
cd frontend-group && npm run dev -- --open
```

Durante desarrollo, `frontend-group/vite.config.js` tiene proxy:

```
/api -> http://localhost:10000
```

Ademas, `apiBase.js` fuerza las llamadas API a `http://localhost:10000` cuando `import.meta.env.DEV` es verdadero.

Variables de entorno reconocidas

Variable	Uso
<code>PORT</code>	Puerto del servidor Express
<code>LCC_DOCS_URL</code>	URL de documentacion Postman v1 de <code>citys-stats</code>
<code>LCC_DOCS_V2_URL</code>	URL de documentacion Postman v2 de <code>citys-stats</code>

8. Scripts, tests y validaciones

Scripts principales de `package.json`:

Script	Comando real	Uso
<code>npm start</code>	<code>node index.js</code>	Arrancar servidor
<code>npm run build</code>	<code>cd frontend-group && npm run build</code>	Compilar Svelte a <code>public</code>
<code>npm run dev-front</code>	<code>cd frontend-group && npm run dev -- --open</code>	Desarrollo frontend
<code>npm run test-citys-stats</code>	Newman LCC v1 local	API ciudades v1
<code>npm run test-citys-stats-v2</code>	Newman LCC v2 local	API ciudades v2
<code>npm run test-LCC</code>	Arranca servidor y ejecuta Newman LCC v1	Validacion LCC v1
<code>npm run test-LCC-v2</code>	Arranca servidor y ejecuta Newman LCC v2	Validacion LCC v2
<code>npm run test-LCC-e2e</code>	Playwright con <code>tests/LCC/playwright.config.js</code>	Flujo navegador LCC
<code>npm test</code>	Suite por defecto del repositorio	Validacion completa del grupo

Comandos recomendados antes de defender:

```
npm.cmd run build
npm.cmd start
```

En otra terminal, si se quiere validar:

```
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Tests LCC e2e cubren:

- Portada con tarjeta de Luis y enlaces LCC.
- Listado de `citys-stats`.
- Creacion de ciudad.
- Borrado individual.
- Borrado total.
- Edicion en `/citys-stats/editar/:city/:country`.
- Busqueda con filtros, orden y limite.

9. Rutas utiles para abrir

Local

```
http://localhost:10000/
http://localhost:10000/citys-stats
http://localhost:10000/analytics
http://localhost:10000/analytics/citys-stats
http://localhost:10000/analytics/citys-stats/map
http://localhost:10000/integrations
http://localhost:10000/integrations/citys-stats
http://localhost:10000/api/v2/citys-stats
http://localhost:10000/api/v1/citys-stats
```

Desplegado

```
https://sos2526-29.onrender.com/
https://sos2526-29.onrender.com/citys-stats
https://sos2526-29.onrender.com/analytics
https://sos2526-29.onrender.com/analytics/citys-stats
https://sos2526-29.onrender.com/analytics/citys-stats/map
https://sos2526-29.onrender.com/integrations
https://sos2526-29.onrender.com/integrations/citys-stats
https://sos2526-29.onrender.com/api/v2/citys-stats/docs
```

Orden recomendado para la defensa LCC:

Orden	Ruta	Que demostrar
1	<code>/</code>	Portada, miembros, enlaces de APIs y documentacion
2	<code>/api/v2/citys-stats/docs</code>	Documentacion Postman de API v2
3	<code>/citys-stats</code>	CRUD completo, filtros, ordenacion y paginacion
4	<code>/analytics/citys-stats</code>	Highcharts individual no lineal
5	<code>/analytics/citys-stats/map</code>	Mapa geoespacial
6	<code>/integrations</code>	Entrada comun de integraciones
7	<code>/integrations/citys-stats</code>	7 integraciones por pais con proxy
8	<code>/analytics</code>	Widget grupal unico

Comprobaciones rapidas con PowerShell:

```
Invoke-WebRequest http://localhost:10000/ -UseBasicParsing
Invoke-WebRequest http://localhost:10000/citys-stats -UseBasicParsing
Invoke-WebRequest http://localhost:10000/analytics -UseBasicParsing
Invoke-WebRequest http://localhost:10000/integrations/citys-stats -UseBasicParsing
Invoke-WebRequest "http://localhost:10000/api/v1/citys-stats/integrations/summary?limit=8" -
UseBasicParsing
```

10. Modelo de datos

citys-stats

Ejemplo:

```
{
  "city": "tokyo",
  "country": "japan",
  "un_2025_population": 33412512
}
```

Clave del recurso:

city + country

Campos:

Campo	Tipo esperado	Significado
city	texto	Ciudad
country	texto	Pais
un_2025_population	entero positivo	Poblacion estimada por Naciones Unidas para 2025

Datos iniciales principales:

jakarta, dhaka, tokyo, delhi, shanghai, guangzhou, cairo, manila, kolkata, seoul, karachi, mumbai

11. Backend: arranque y responsabilidades

Archivo principal:

index.js

Responsabilidades:

1. Importa Express, path, NeDB y CORS.
2. Crea `app`.

3. Define `port` .
4. Activa `cors()` .
5. Activa `express.json()` para leer cuerpos JSON.
6. Crea tres bases NeDB:

- `naturalDisastersDb`
- `citysStatsDb`
- `wineStatsDb`

1. Carga y registra modulos de API:

- `src/back/v1/natural-disasters.js`
- `src/back/v2/natural-disasters.js`
- `src/back/v1/citys-stats.js`
- `src/back/v2/citys-stats.js`
- `src/back/v1/wine-stats.js`

1. Sirve el frontend compilado desde `public` .
2. Define `/` y `/about` para devolver el frontend. En la SPA actual, `/about` cae en la portada porque no hay una pantalla `routes/about` .
3. Define proxy `/api/proxy/exportations-stats` .
4. Define fallback para toda ruta que no empiece por `/api` .
5. Lanza `app.listen` .

Frase de defensa:

`index.js` es el punto donde se juntan las capas. Crea el servidor, abre las bases, registra las APIs REST y sirve la SPA de Svelte desde `public` .

Bases de datos

Variable	Archivo
<code>naturalDisastersDb</code>	<code>src/back/natural-disasters.db</code>
<code>citysStatsDb</code>	<code>src/back/citys-stats.db</code>
<code>wineStatsDb</code>	<code>src/back/wine-stats.db</code>

Todas usan:

```
autoload: true
```

Eso abre el archivo al arrancar.

Fallback de la SPA

Esta ruta es clave:

```
app.get(/^\/(?!api\/).*/ , (request, response) => {
  response.sendFile(frontendIndexPath);
});
```

Significa:

- Si la ruta no empieza por `/api`, se devuelve `public/index.html`.
- Svelte decide despues que pantalla mostrar.
- Por eso funcionan rutas directas como `/analytics` o `/citys-stats/editar/tokyo/japan`.

12. APIs REST del proyecto

12.1 `citys-stats` v2

Archivo:

```
src/back/v2/citys-stats.js
```

Ruta base:

```
/api/v2/citys-stats
```

Uso principal:

- CRUD de LCC desde el frontend.
- Búsqueda libre.
- Filtros exactos.
- Ordenacion.
- Paginacion.
- Validacion estricta.

Endpoints:

Metodo	Ruta	Resultado
GET	<code>/api/v2/citys-stats/docs</code>	Redirige a Postman
GET	<code>/api/v2/citys-stats/loadInitialData</code>	Inserta datos si la base esta vacia o devuelve existentes
GET	<code>/api/v2/citys-stats</code>	Lista con filtros, <code>q</code> , <code>sort</code> , <code>offset</code> , <code>limit</code>
GET	<code>/api/v2/citys-stats/:city/:country</code>	Obtiene un registro
POST	<code>/api/v2/citys-stats</code>	Crea un registro
POST	<code>/api/v2/citys-stats/:city/:country</code>	405

Metodo	Ruta	Resultado
PUT	/api/v2/citys-stats	405
PUT	/api/v2/citys-stats/:city:country	Actualiza registro
DELETE	/api/v2/citys-stats	Borra todos y devuelve 204
DELETE	/api/v2/citys-stats/:city:country	Borra uno y devuelve 204

Queries utiles:

```
GET /api/v2/citys-stats?q=india
GET /api/v2/citys-stats?city=tokyo
GET /api/v2/citys-stats?country=china
GET /api/v2/citys-stats?un_2025_population=33412512
GET /api/v2/citys-stats?sort=-un_2025_population
GET /api/v2/citys-stats?sort=city&offset=0&limit=5
```

12.2 citys-stats v1

Archivo:

```
src/back/v1/citys-stats.js
```

Ruta base:

```
/api/v1/citys-stats
```

Uso principal:

- Compatibilidad v1.
- CRUD basico con filtros exactos y paginacion.
- Endpoints de agregacion.
- Proxies e integraciones externas.

Endpoints especiales:

Metodo	Ruta	Uso
GET	/api/v1/citys-stats/top-cities	Top por poblacion
GET	/api/v1/citys-stats/country-summaries	Agregado local por pais
GET	/api/v1/citys-stats/integrations/geocoding/:city	Proxy Open-Meteo
GET	/api/v1/citys-stats/integrations/country/:country	Proxy REST Countries
GET	/api/v1/citys-stats/integrations/world-bank/:countryCode	Proxy World Bank
GET	/api/v1/citys-stats/integrations/sos-tourist-arrivals	Proxy SOS2526-25
GET	/api/v1/citys-stats/integrations/sos-earthquakes	Proxy SOS2526-19
GET	/api/v1/citys-stats/integrations/sos-fifa-squad-values	Proxy SOS2526-26
GET	/api/v1/citys-stats/integrations/sos-esports-earnings	Proxy SOS2526-30

Metodo	Ruta	Uso
GET	/api/v1/citys-stats/integrations/summary	Resumen integrado

13. Frontend: rutas, servicios y pantallas

Punto de entrada

Archivo:

```
frontend-group/src/main.js
```

Hace:

1. Importa `mount` de Svelte.
2. Importa `app.css`.
3. Importa `App.svelte`.
4. Monta la app en `document.getElementById("app")`.

Router real

Archivo:

```
frontend-group/src/App.svelte
```

Funcionamiento:

1. Usa `import.meta.glob("./routes/**/*.svelte", { eager: true })`.
2. Convierte cada ruta de archivo en ruta web.
3. Compila segmentos dinamicos como `[city]` o `[country]` a expresiones regulares.
4. Lee `window.location.pathname`.
5. Elige el componente correspondiente.
6. Escucha `popstate` para actualizar la pantalla.

Ejemplos:

Archivo	Ruta
<code>routes/+page.svelte</code>	<code>/</code>
<code>routes/citys-stats/+page.svelte</code>	<code>/citys-stats</code>
<code>routes/citys-stats/editar/[city]/[country]/+page.svelte</code>	<code>/citys-stats/editar/:city/:country</code>
<code>routes/analytics/+page.svelte</code>	<code>/analytics</code>
<code>routes/integrations/citys-stats/+page.svelte</code>	<code>/integrations/citys-stats</code>

Navegacion interna

Archivo:

```
frontend-group/src/lib/navigation.js
```

Funciones:

- `navigate(path)` : usa `history.pushState` y dispara `PopStateEvent` .
- `replace(path)` : reemplaza la URL actual y dispara `PopStateEvent` .
- `back()` : llama a `window.history.back()` .

Services

Los services evitan duplicar URLs y parsing de errores en cada pantalla.

Service	API que llama	Uso
<code>apiBase.js</code>	Calcula origen	Local con Vite o despliegue
<code>citysStatsApi.js</code>	<code>/api/v2/citys-stats</code>	CRUD LCC
<code>citysStatsIntegrations.js</code>	<code>/api/v1/citys-stats</code>	Integraciones LCC

Pantallas principales

Ruta	Archivo
<code>/</code>	<code>frontend-group/src/routes/+page.svelte</code>
<code>/citys-stats</code>	<code>frontend-group/src/routes/citys-stats/+page.svelte</code>
<code>/citys-stats/editar/:city/:country</code>	<code>frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte</code>
<code>/analytics</code>	<code>frontend-group/src/routes/analytics/+page.svelte</code>
<code>/analytics/citys-stats</code>	<code>frontend-group/src/routes/analytics/citys-stats/+page.svelte</code>
<code>/analytics/citys-stats/map</code>	<code>frontend-group/src/routes/analytics/citys-stats/map/+page.svelte</code>
<code>/integrations</code>	<code>frontend-group/src/routes/integrations/+page.svelte</code>
<code>/integrations/citys-stats</code>	<code>frontend-group/src/routes/integrations/citys-stats/+page.svelte</code>

14. Parte LCC: citys-stats

Que es

`citys-stats` gestiona estadisticas de poblacion estimada para ciudades en 2025.

Campos:

```
city
country
un_2025_population
```

Identificador:

```
city + country
```

Por que se llama `citys-stats`

Aunque en ingles correcto seria `cities-stats`, el recurso publicado en la practica es `citys-stats`.

Cambiarlo romperia:

- URLs.
- Tests.
- Documentacion Postman.
- Enlaces del frontend.
- Integraciones ya entregadas.

Frase de defensa:

Mantengo `citys-stats` porque es el contrato publico del proyecto. En APIs, estabilidad del contrato pesa mas que corregir un nombre historico.

API LCC principal

Para CRUD se usa:

```
/api/v2/citys-stats
```

Motivo:

- Tiene busqueda libre `q`.
- Tiene ordenacion `sort`.
- Tiene paginacion `limit` y `offset`.
- Tiene validacion estricta.
- Devuelve JSON limpio sin `_id`.

API LCC de integraciones

Para integraciones se usa:

```
/api/v1/citys-stats/integrations/...
```

Motivo:

- El proxy externo esta implementado en v1.
- Mantiene compatibilidad con entregas previas.
- Centraliza llamadas a Open-Meteo, REST Countries, World Bank y APIs SOS externas.

CRUD visible

Pantalla:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Funciones principales de la pantalla:

- `emptycreateForm`
- `emptySearchForm`
- `clearFeedback`
- `hasQueryValues`
- `parsePositiveInteger`
- `parseOptionalNonNegativeInteger`
- `parseOptionalPositiveInteger`
- `validateCityStatForm`
- `buildSearchQuery`
- `refreshList`
- `handleSearch`
- `handleResetSearch`
- `handleCreate`
- `handleLoadInitialData`
- `handleDeleteAll`
- `handleDeleteOne`
- `openEdit`

Edicion separada

Pantalla:

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

Comportamiento importante:

- Carga el registro con `getOneCityStat(params.city, params.country)`.
- Si el usuario cambia solo poblacion, hace `PUT`.
- Si el usuario cambia `city` o `country`, crea el registro nuevo y borra el antiguo.
- Usa `replace(...)` para actualizar la URL cuando cambia la clave.

Esto evita una limitacion natural de `PUT`: la URL identifica el recurso original, pero si cambia la clave compuesta, en realidad nace otra URL.

Visualizacion individual

Pantalla:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
```

Usa:

- Highcharts.
- Grafico `pie`.
- Datos de `getAllCityStats({ sort: "-un_2025_population" })`.

Frase:

La visualizacion individual no es `line`; es un `pie` que representa la proporcion de poblacion estimada por ciudad.

Mapa

Pantalla:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Usa:

- Highcharts Maps.
- `@highcharts/map-collection/custom/world.topo.json`.
- Coordenadas locales en el objeto `coordinates`.
- Marcadores `mappoint`.
- Color y radio segun poblacion.

Claves de coordenadas:

```
city|country
```

Ejemplo:

```
"tokyo|japan": { lat: 35.6762, lon: 139.6503 }
```

15. Integraciones LCC

Pantalla:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Backend:

```
src/back/v1/citys-stats.js
```

Service:

```
frontend-group/src/services/citysStatsIntegrations.js
```

Decision clave: integrar por pais

La integracion se hace por `country` , no por ciudad.

Motivos:

- REST Countries trabaja por pais.
- World Bank trabaja por pais o codigo ISO.
- Muchas APIs SOS externas publican pais o ISO3.
- Los nombres de ciudad no coinciden bien entre fuentes.
- El cruce por pais produce menos huecos y mas informacion real.

Frase de defensa:

Integro por pais porque es el campo comun mas estable entre mi recurso y las APIs externas. Asi evito una integracion fragil por nombres de ciudades y consigo widgets con datos reales comparables.

APIs integradas

API	Tipo	Fuente	Endpoint proxy local	Widget
Open-Meteo Geocoding	No SOS	<code>https://geocoding-api.open-meteo.com/v1/search</code>	<code>/api/v1/citys-stats/integrations/geocoding/:city</code>	<code>treemap</code>
REST Countries	No SOS	<code>https://restcountries.com/v3.1/name/:country</code>	<code>/api/v1/citys-stats/integrations/country/:country</code>	<code>sankey</code>
World Bank Indicators	No SOS	<code>https://api.worldbank.org/v2/country/:code/indicator/SP.POP.TOTL</code>	<code>/api/v1/citys-stats/integrations/world-bank/:countryCode</code>	<code>lollipop</code>
SOS2526-25 <code>international-tourist-arrivals</code>	Alumno SOS	<code>https://sos2526-25.onrender.com/api/v2/international-tourist-arrivals</code>	<code>/api/v1/citys-stats/integrations/sos-tourist-arrivals</code>	<code>variwide</code>
SOS2526-19 <code>earthquakes</code>	Alumno SOS	<code>https://sos2526-19.onrender.com/api/v1/earthquakes</code>	<code>/api/v1/citys-stats/integrations/sos-earthquakes</code>	<code>bullet</code>
SOS2526-26 <code>fifa-squad-value-per-years</code>	Alumno SOS	<code>https://sos2526-26.onrender.com/api/v2/fifa-squad-</code>	<code>/api/v1/citys-stats/integrations/sos-fifa-</code>	<code>dumbbell</code>

API	Tipo	Fuente	Endpoint proxy local	Widget
		value-per-years	squad-values	
SOS2526-30 esportsearnings-stats	Alumno SOS	https://sos2526-30.onrender.com/api/v1/esportsearnings-stats	/api/v1/citys-stats/integrations/sos-esports-earnings	sunburst

Cumplimiento D03

- Usa 7 APIs externas.
- Usa 3 APIs no SOS: Open-Meteo, REST Countries y World Bank.
- Usa 4 APIs SOS de otros grupos: 25, 19, 26 y 30.
- Todas devuelven JSON.
- Todas pasan por proxy propio en Express.
- No se muestra JSON crudo: se transforma en graficas, tarjetas, tablas y metricas.
- Los widgets no son `line`.
- La vista esta enlazada desde `/integrations`.
- El widget grupal esta en `/analytics`.

Flujo de integraciones

1. El usuario abre `/integrations/citys-stats`.
2. La pantalla llama a `getCountrySummaries(selectedLimit)`.
3. El service hace `GET /api/v1/citys-stats/country-summaries?limit=N`.
4. El backend lee NeDB.
5. `buildCityCountrySummaries` agrega ciudades por pais.
6. La pantalla usa esos paises como base.
7. Para cada pais o ciudad principal, llama a endpoints proxy.
8. Express llama a APIs externas con `fetchJson`.
9. El backend normaliza datos y controla errores.
10. El frontend construye widgets Highcharts distintos.

Endpoint resumen

Ruta:

```
GET /api/v1/citys-stats/integrations/summary?limit=8
```

Respuesta real resumida:

```
{
  localResource: "/api/v1/citys-stats/country-summaries",
  externalApis: string[],
  studentApis: object[],
  count: number,
}
```

```
items: object[]
}
```

Campos importantes de cada item:

Campo	Significado
<code>city</code>	Ciudad principal del pais
<code>country</code>	Pais usado como clave de cruce
<code>cityCount</code>	Numero de ciudades locales de ese pais
<code>topCity</code>	Ciudad local mas poblada
<code>topCityPopulation</code>	Poblacion de la ciudad local mas poblada
<code>cities</code>	Lista de ciudades locales del pais
<code>un_2025_population</code>	Suma local de poblacion 2025 del pais
<code>geocoding</code>	Datos de Open-Meteo
<code>countryInfo</code>	Datos de REST Countries
<code>worldBankPopulation</code>	Dato de poblacion World Bank
<code>touristArrivals</code>	Datos agregados de turismo
<code>earthquakeStats</code>	Datos agregados de terremotos
<code>fifaSquadValue</code>	Datos agregados FIFA
<code>esportsEarnings</code>	Datos agregados eSports
<code>integrationErrors</code>	Errores parciales

16. Funciones importantes explicadas

Esta seccion es la guia tecnica principal para defender y modificar la parte LCC `citys-stats`. La idea no es memorizar todo el repositorio, sino saber leer una funcion, reconocer en que capa esta, explicar por que existe y saber que tocar si en defensa piden un cambio.

Orden recomendado para estudiar esta seccion:

1. Leer el mapa por capas para saber donde vive cada cosa.
2. Aprender las fichas de backend v2, porque son el contrato REST principal.
3. Aprender services y pantallas, porque explican como llega la accion del usuario al backend.
4. Repasar integraciones, mapa y analytics, porque ahí suelen preguntar por asincronia y APIs externas.
5. Usar las recetas de cambio como chuleta para modificar el programa en directo.

16.1 Como defender cualquier funcion

Si el profesor senala una funcion, contesta siempre con este orden:

1. Archivo y capa: backend, service, pantalla, grafica o integracion.
2. Momento de ejecucion: al arrancar, al abrir pantalla, al pulsar boton o al recibir una peticion.
3. Entrada: parametros, req.query, req.params, req.body, estado Svelte o respuesta externa.
4. Trabajo: valida, transforma, consulta base, llama fetch, pinta grafica o actualiza estado.
5. Salida: JSON, codigo HTTP, valor devuelto, Error lanzado o variables de pantalla actualizadas.
6. Cambio probable: que linea tocaria si me piden cambiar el comportamiento.

Frase comodin segura:

Esta funcion tiene una responsabilidad concreta dentro de su capa. Recibe datos, los valida o transforma, llama a la siguiente capa si hace falta y devuelve un resultado controlado. Si se cambia, hay que revisar las funciones que la llaman y los tests asociados.

16.2 Mapa de funciones LCC por capa

Capa	Archivo	Funciones o bloques que debes reconocer
Arranque comun	index.js	creacion de <code>app</code> , bases NeDB, registro de APIs, <code>express.static</code> , fallback SPA
Backend CRUD v2	src/back/v2/citys-stats.js	<code>removeDatabaseId</code> , <code>hasExactCityFields</code> , <code>normalizeCityStat</code> , handlers <code>GET</code> , <code>POST</code> , <code>PUT</code> , <code>DELETE</code>
Backend integraciones v1	src/back/v1/citys-stats.js	<code>parseLimit</code> , <code>normalizeCountryKey</code> , <code>buildCityCountrySummaries</code> , <code>fetchJson</code> , <code>safeExternal</code> , <code>buildIntegratedCity</code>
Service base	frontend-group/src/services/apiBase.js	<code>API_ORIGIN</code> , <code>apiPath</code>
Service CRUD	frontend-group/src/services/citysStatsApi.js	<code>buildUrl</code> , <code>encodePathValue</code> , <code>friendlyApiMessage</code> , <code>handleResponse</code> , <code>getAllCitysStats</code> , <code>createCityStat</code> , <code>updateCityStat</code>
Service integraciones	frontend-group/src/services/citysStatsIntegrations.js	<code>handleResponse</code> , <code>getCountrySummaries</code> , <code>getGeocoding</code> , <code>getCountryInfo</code> , <code>getWorldBankPopulation</code> , <code>getSos...</code>
CRUD Svelte	frontend-group/src/routes/citys-stats/+page.svelte	<code>validateCityStatForm</code> , <code>buildSearchQuery</code> , <code>refreshList</code> , <code>handleCreate</code> , <code>handleSearch</code> , <code>openEdit</code>
Edicion Svelte	frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte	<code>loadResource</code> , <code>isSameResource</code> , <code>handleUpdate</code> , <code>updateRoute</code>
Analytics LCC	frontend-group/src/routes/analytics/citys-stats/+page.svelte	<code>loadHighcharts</code> , <code>renderChart</code> , <code>loadAnalytics</code>
Mapa LCC	frontend-group/src/routes/analytics/citys-stats/map/+page.svelte	<code>keyFor</code> , <code>colorFor</code> , <code>radiusFor</code> , <code>renderMap</code> , <code>loadMapData</code>
Integraciones UI	frontend-group/src/routes/integrations/citys-stats/+page.svelte	<code>safeLoad</code> , <code>collectError</code> , <code>loadHighcharts</code> , <code>createChart</code> , <code>renderIntegrationCharts</code> , <code>loadIntegrations</code>

16.3 Backend CRUD v2: contrato principal

Archivo principal:

```
src/back/v2/citys-stats.js
```

Esta es la API principal de LCC. Se usa para crear, listar, buscar, editar y borrar `citys-stats`.

`removeDatabaseId`

Codigo:

```
function removeDatabaseId(doc) {  
  if (!doc) return doc;  
  const { _id, ...rest } = doc;  
  return rest;  
}
```

Que hace:

Quita `_id`, que es un campo interno de NeDB, antes de devolver el JSON al cliente. La API publica solo debe exponer `city`, `country` y `un_2025_population`.

Como te pueden pedir cambiarla:

- Si quieren ocultar otro campo interno, se anade al destructuring: `const { _id, _rev, ...rest } = doc`.
- Si quieren devolver `_id` para depurar, se deja de usar esta funcion en las respuestas, aunque no es recomendable para el contrato publico.

Respuesta de defensa:

NeDB necesita `_id` para guardar documentos, pero ese campo no forma parte del contrato REST de `citys-stats`.

`hasExactCityFields`

Codigo:

```
function hasExactCityFields(body) {  
  if (!body || typeof body !== "object" || Array.isArray(body)) return false;  
  
  const expected = ["city", "country", "un_2025_population"].sort();  
  const keys = Object.keys(body).sort();  
  
  return keys.length === expected.length &&  
    keys.every((k, i) => k === expected[i]);  
}
```

Que hace:

Comprueba que el body tenga exactamente los campos permitidos. No acepta campos de menos ni campos de mas.

Como te pueden pedir cambiarla:

- Anadir `continent`: meterlo en `expected`.
- Hacer un campo opcional: separar campos obligatorios y opcionales en vez de exigir igualdad exacta.
- Aceptar campos extra: quitar la comparacion estricta, aunque se pierde control del contrato.

Pillada probable:

Si mando `{ city, country, un_2025_population, extra }`, devuelve `400` porque la API valida estructura exacta.

`normalizeCityStat`

Codigo:

```
function normalizeCityStat(body) {
  if (!hasExactCityFields(body)) return null;

  const city = String(body.city).trim().toLowerCase();
  const country = String(body.country).trim().toLowerCase();
  const un_2025_population = Number(body.un_2025_population);

  if (
    !city ||
    !country ||
    !Number.isInteger(un_2025_population) ||
    un_2025_population <= 0
  ) {
    return null;
  }

  return { city, country, un_2025_population };
}
```

Que hace:

Valida contenido y normaliza valores: textos sin espacios, en minusculas, y poblacion convertida a entero positivo.

Como te pueden pedir cambiarla:

- Permitir poblacion `0`: cambiar `un_2025_population <= 0` por `un_2025_population < 0`.
- Mantener mayusculas originales: quitar `.toLowerCase()`, pero revisar busquedas, URLs y tests.
- Anadir un campo: leerlo, validarlo y devolverlo en el objeto final.

Respuesta de defensa:

El frontend tambien valida, pero esta funcion es la barrera real. Aunque llamen a la API sin interfaz, el backend no guarda datos invalidos.

Handler `GET /api/v2/citys-stats`

Fragmento clave 1, lectura, filtros y ordenacion:

```
app.get(BASE_API_URL, (req, res) => {
  db.find({}, (err, docs) => {
    if (err) return res.sendStatus(500);

    let result = docs.map(removeDatabaseId);

    if (req.query.city !== undefined) {
      result = result.filter(
        d => d.city === String(req.query.city).trim().toLowerCase()
      );
    }

    if (req.query.country !== undefined) {
      result = result.filter(
        d => d.country === String(req.query.country).trim().toLowerCase()
      );
    }

    if (req.query.un_2025_population !== undefined) {
      const value = Number(req.query.un_2025_population);
      if (!Number.isFinite(value)) {
        return res.status(400).json({ error: "Invalid query" });
      }
      result = result.filter(d => d.un_2025_population === value);
    }

    if (req.query.q !== undefined) {
      const q = String(req.query.q).trim().toLowerCase();
      result = result.filter(d =>
        d.city.includes(q) || d.country.includes(q)
      );
    }
  });
});
```

Fragmento clave 2, ordenacion y paginacion dentro del mismo handler:

```
if (req.query.sort !== undefined) {
  const sort = String(req.query.sort).trim();
  let field = sort;
  let direction = 1;

  if (sort.startsWith("-")) {
    field = sort.slice(1);
    direction = -1;
  }

  const allowedFields = ["city", "country", "un_2025_population"];
```

```

    if (!allowedFields.includes(field)) {
      return res.status(400).json({ error: "Invalid sort field" });
    }

    result.sort((a, b) => {
      if (a[field] < b[field]) return -1 * direction;
      if (a[field] > b[field]) return 1 * direction;
      return 0;
    });
  }

  let offset = 0;
  let limit = result.length;

  if (req.query.offset !== undefined) {
    offset = Number(req.query.offset);
    if (!Number.isInteger(offset) || offset < 0) {
      return res.status(400).json({ error: "Invalid offset" });
    }
  }

  if (req.query.limit !== undefined) {
    limit = Number(req.query.limit);
    if (!Number.isInteger(limit) || limit < 0) {
      return res.status(400).json({ error: "Invalid limit" });
    }
  }

  return res.status(200).json(result.slice(offset, offset + limit));

```

Que hace:

Lee todos los documentos, quita `_id`, aplica filtros exactos, busqueda libre, ordenacion y paginacion.

Orden real:

1. `db.find({})` lee NeDB.
2. `removeDatabaseId` limpia respuesta.
3. Filtra por `city`, `country` y `un_2025_population`.
4. Aplica `q` sobre ciudad o pais.
5. Ordena con lista blanca de campos.
6. Aplica `offset` y `limit`.
7. Devuelve `200` con array JSON.

Como te pueden pedir cambiarla:

- Filtro `min_population`: anadir bloque con `Number(req.query.min_population)` y `>=`.
- Ordenar por campo nuevo: anadirlo a `allowedFields`.
- Buscar tambien por otro campo: meterlo dentro del filtro de `q`.
- Cambiar paginacion por defecto: tocar `offset` y `limit`.

Pillada probable:

Los query params llegan como texto, por eso se convierten y validan antes de filtrar.

Handler `POST /api/v2/citys-stats`

Codigo:

```
app.post(BASE_API_URL, (req, res) => {
  const item = normalizeCityStat(req.body);

  if (!item) {
    return res.status(400).json({
      error: "JSON body does not match expected structure"
    });
  }

  db.findOne({ city: item.city, country: item.country }, (err, doc) => {
    if (err) return res.sendStatus(500);
    if (doc) return res.status(409).json({ error: "Resource already exists" });

    db.insert(item, (err2, newDoc) => {
      if (err2) return res.sendStatus(500);
      return res.status(201).json(removeDatabaseId(newDoc));
    });
  });
});
```

Que hace:

Crea un registro nuevo si el body es valido y no existe ya la pareja `city + country`.

Como te pueden pedir cambiarla:

- Cambiar la clave de duplicado: modificar el objeto de `db.findOne`.
- Devolver otro codigo al crear: cambiar `res.status(201)`.
- Guardar un campo nuevo: primero debe salir de `normalizeCityStat`.

Respuesta de defensa:

`POST` crea dentro de la coleccion. Si ya existe el recurso, devuelve `409 Conflict`.

Handler `PUT /api/v2/citys-stats/:city/:country`

Fragmento clave:

```
app.put(`${BASE_API_URL}/${city}/${country}`, (req, res) => {
  const city = req.params.city.trim().toLowerCase();
  const country = req.params.country.trim().toLowerCase();

  const item = normalizeCityStat(req.body);

  if (!item) {
    return res.status(400).json({
```

```

        error: "JSON body does not match expected structure"
    });
}

if (item.city !== city || item.country !== country) {
    return res.status(400).json({ error: "URL and body do not match" });
}

db.findOne({ city, country }, (err, doc) => {
    if (err) return res.sendStatus(500);
    if (!doc) return res.status(404).json({ error: "Resource not found" });

    db.update({ city, country }, item, {}, (err2) => {
        if (err2) return res.sendStatus(500);
        db.findOne({ city, country }, (err3, updated) => {
            if (err3) return res.sendStatus(500);
            return res.status(200).json(removeDatabaseId(updated));
        });
    });
});
});
});

```

Que hace:

Actualiza un recurso identificado por `city + country`. Exige que la URL y el body coincidan para evitar modificar un recurso distinto al solicitado.

Como te pueden pedir cambiarla:

- Permitir cambiar ciudad o país con `PUT`: quitar la comprobación URL/body y revisar duplicados. En este proyecto se resuelve desde frontend creando nuevo y borrando antiguo.
- Cambiar clave del recurso: tocar parámetros de ruta, `db.findOne`, `db.update`, `deleteCityStat`, `getOneCityStat` y rutas de edición.
- Devolver `204`: cambiar respuesta final, pero pierdes el objeto actualizado.

Respuesta de defensa:

`PUT` actualiza un recurso concreto. La URL identifica el recurso y el body debe describir ese mismo recurso.

Handlers `DELETE`

Código:

```

app.delete(BASE_API_URL, (req, res) => {
    db.remove({}, { multi: true }, (err) => {
        if (err) return res.sendStatus(500);
        return res.sendStatus(204);
    });
});

app.delete(`${BASE_API_URL}/:city/:country`, (req, res) => {
    const city = req.params.city.trim().toLowerCase();

```

```
const country = req.params.country.trim().toLowerCase();

db.remove({ city, country }, {}, (err, numRemoved) => {
  if (err) return res.sendStatus(500);
  if (numRemoved === 0) {
    return res.status(404).json({ error: "Resource not found" });
  }

  return res.sendStatus(204);
});
});
```

Que hacen:

El primer `DELETE` borra toda la coleccion. El segundo borra un registro concreto.

Como te pueden pedir cambiarlos:

- Devolver el elemento borrado: habria que buscarlo antes de borrar.
- Prohibir borrado total: cambiar el handler de coleccion a `405` o eliminar el boton del frontend.
- Borrado logico: anadir campo `deleted: true` en vez de `db.remove`.

16.4 Backend integraciones v1

Archivo principal:

```
src/back/v1/citys-stats.js
```

Aunque el CRUD principal esta en v2, las integraciones de LCC estan en v1 porque ahi se concentraron los endpoints proxy hacia APIs externas.

`parseLimit`

Codigo:

```
function parseLimit(value, fallback, max) {
  if (value === undefined) return fallback;

  const limit = Number(value);

  if (!Number.isInteger(limit) || limit < 1 || limit > max) {
    return null;
  }

  return limit;
}
```

Que hace:

Valida cuantos resultados puede pedir una integracion. Evita valores negativos, decimales, texto o limites demasiado grandes.

Como te pueden pedir cambiarla:

- Permitir mas elementos: subir `max` en la llamada, no necesariamente en la funcion.
- Permitir `0`: cambiar `limit < 1` por `limit < 0`, aunque para integraciones normalmente no tiene sentido.
- Cambiar valor por defecto: tocar el `fallback` en cada endpoint.

`normalizeCountryKey`

Codigo:

```
function normalizeCountryKey(value) {
  return String(value ?? "")
    .trim()
    .normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "")
    .replace(/[-_]+/g, " ")
    .toLowerCase();
}
```

Que hace:

Convierte paises a una clave comparable aunque lleguen con mayusculas, tildes, guiones o guiones bajos.

Como te pueden pedir cambiarla:

- Si una API usa nombres raros, anadir una tabla de equivalencias antes de devolver la clave.
- Si se quiere integrar por ciudad, crear una funcion equivalente para ciudades y revisar todos los cruces por pais.

Respuesta de defensa:

Las APIs externas no escriben paises exactamente igual. Normalizar reduce fallos de coincidencia.

`buildCityCountrySummaries`

Fragmento clave:

```
function buildCityCountrySummaries(items) {
  const byCountry = new Map();

  items.forEach((item) => {
    const key = normalizeCountryKey(item.country);
    if (!key) return;

    const population = readFiniteNumber(item.un_2025_population, 0);
    const current = byCountry.get(key) || {
      country: item.country,
      countryKey: key,
      city: item.city,
      topCity: item.city,
      topCityPopulation: population,
    };
  });
}
```

```

        cityCount: 0,
        un_2025_population: 0,
        cities: []
    };

    current.cityCount += 1;
    current.un_2025_population += population;
    current.cities.push({ city: item.city, population });

    if (population > current.topCityPopulation) {
        current.city = item.city;
        current.topCity = item.city;
        current.topCityPopulation = population;
    }

    byCountry.set(key, current);
});

return [...byCountry.values()]
    .map((item) => ({
        ...item,
        cities: item.cities.sort((a, b) => b.population - a.population)
    }))
    .sort((a, b) => b.un_2025_population - a.un_2025_population);
}

```

Que hace:

Agrupar ciudades locales por país. Calcula cuántas ciudades hay, población agregada y ciudad principal. Es la base para cruzar con REST Countries, World Bank y APIs SOS.

Como te pueden pedir cambiarla:

- Calcular media: añadir `averagePopulation = un_2025_population / cityCount`.
- Integrar por ciudad: esta función dejaría de ser la base y habría que cambiar endpoints y widgets.
- Mostrar todos los países sin ordenar: quitar o cambiar el `sort` final.

fetchJson

Código:

```

async function fetchJson(url, sourceName, timeoutMs = 20000) {
    const controller = new AbortController();
    const timeout = setTimeout(() => controller.abort(), timeoutMs);

    try {
        const response = await fetch(url, {
            headers: {
                Accept: "application/json",
                "User-Agent": "SOS2526-29 citys-stats integration"
            },
            signal: controller.signal
        });
    }

    const text = await response.text();
    let data = null;

```

```

    try {
      data = text ? JSON.parse(text) : null;
    } catch {
      throw new Error(`${sourceName} did not return JSON`);
    }

    if (!response.ok) {
      const reason = data?.message || data?.error || response.statusText;
      throw new Error(`${sourceName} returned ${response.status}: ${reason}`);
    }

    return data;
  } finally {
    clearTimeout(timeout);
  }
}

```

Que hace:

Llama a APIs externas con timeout, cabeceras, lectura de texto, parseo JSON y control de errores HTTP.

Como te pueden pedir cambiarla:

- Aumentar timeout: cambiar el tercer parametro o el valor por defecto.
- Consumir API con token: anadir cabecera `Authorization`.
- Aceptar CSV o texto: cambiar la parte de `JSON.parse`.

Respuesta de defensa:

Centralizar `fetchJson` evita repetir control de timeout y errores en cada integracion.

`safeExternal`

Codigo:

```

async function safeExternal(source, task) {
  try {
    return { source, data: await task(), error: null };
  } catch (err) {
    return { source, data: null, error: err.message };
  }
}

```

Que hace:

Convierte una llamada externa en un objeto controlado. Si falla, no rompe todo el resumen; devuelve `error` para mostrarlo como fallo parcial.

Como te pueden pedir cambiarla:

- Hacer que falle toda la integracion si falla una API: dejar que el error se lance sin envolverlo.

- Guardar mas detalle: anadir `status`, `url` o `timestamp` al objeto devuelto.

`buildIntegratedCity`

Fragmento clave:

```
function buildIntegratedCity(base, worldBankByCode, worldBankBatchError, studentApis) {
  const code = base.countryResult.data?.cca3;
  let worldBankResult;

  if (!code) {
    worldBankResult = {
      source: "World Bank Indicators API",
      data: null,
      error: "Country ISO3 code not available"
    };
  } else if (worldBankBatchError) {
    worldBankResult = {
      source: "World Bank Indicators API",
      data: null,
      error: worldBankBatchError
    };
  } else {
    const data = worldBankByCode.get(code) ?? null;
    worldBankResult = {
      source: "World Bank Indicators API",
      data,
      error: data ? null : "World Bank data not found"
    };
  }

  return {
    city: base.item.city,
    country: base.item.country,
    cityCount: base.item.cityCount ?? 1,
    topCity: base.item.topCity ?? base.item.city,
    topCityPopulation: base.item.topCityPopulation ?? base.item.un_2025_population,
    un_2025_population: base.item.un_2025_population,
    geocoding: base.geocodingResult.data,
    countryInfo: base.countryResult.data,
    worldBankPopulation: worldBankResult.data,
    integrationErrors: [base.geocodingResult, base.countryResult, worldBankResult]
      .filter((result) => result.error)
      .map((result) => ({ source: result.source, error: result.error })),
  };
}
```

Que hace:

Monta el objeto final del endpoint `/api/v1/citys-stats/integrations/summary`: datos locales, geocoding, pais, World Bank y errores parciales.

Como te pueden pedir cambiarla:

- Anadir una API externa nueva al resumen: calcularla antes y meter su dato aqui.

- Cambiar el formato JSON del resumen: cambiar las propiedades devueltas y actualizar service/pantalla.
- Mostrar mas errores parciales: anadir mas resultados a la lista `integrationErrors`.

16.5 Services frontend

Los services separan Svelte de los detalles HTTP. La pantalla no debe construir URLs complejas ni traducir todos los codigos de error.

`apiPath`

Archivo:

```
frontend-group/src/services/apiBase.js
```

Codigo:

```
export const API_ORIGIN = import.meta.env.DEV
  ? "http://localhost:10000"
  : window.location.origin;

export function apiPath(path) {
  return `${API_ORIGIN}${path}`;
}
```

Que hace:

En desarrollo llama al backend Express del puerto `10000`. En produccion usa el mismo origen que la web desplegada.

Como te pueden pedir cambiarla:

- Cambiar puerto local: modificar `http://localhost:10000`.
- Usar variable de entorno: leer `import.meta.env.VITE_API_ORIGIN`.
- Desplegar frontend y backend separados: cambiar `API_ORIGIN` de produccion.

`buildUrl`

Archivo:

```
frontend-group/src/services/citysStatsApi.js
```

Codigo:

```
function buildUrl(path = "", query = {}) {
  const url = new URL(`${CITYS_STATS_API_BASE}${path}`, window.location.origin);

  Object.entries(query).forEach(([key, value]) => {
    if (value === undefined || value === null || value === "") {
      return;
    }
  });
}
```

```

        url.searchParams.set(key, value);
    });

    return url.toString();
}

```

Que hace:

Crea la URL del CRUD y anade query params sin concatenar strings a mano.

Como te pueden pedir cambiarla:

- Pasar de v2 a v3: cambiar `CITYS_STATS_API_BASE`, no esta funcion.
- Parametros repetidos: cambiar `set` por `append`.
- Enviar valores vacios: quitar el filtro de `undefined`, `null` y `""`.

handleResponse

Codigo:

```

async function handleResponse(response) {
    if (response.status === 204) {
        return null;
    }

    const text = await response.text();
    let data = null;

    try {
        data = text ? JSON.parse(text) : null;
    } catch {
        data = text;
    }

    if (!response.ok) {
        throw new Error(friendlyApiMessage(response.status, data?.error));
    }

    return data;
}

```

Que hace:

Convierte respuestas `fetch` en datos o en `Error`. Las pantallas solo necesitan `try/catch`.

Como te pueden pedir cambiarla:

- Cambiar mensajes visibles: tocar `friendlyApiMessage`.
- Mostrar el error exacto del backend: lanzar `data?.error` directamente.
- Soportar otro tipo de respuesta: cambiar la lectura de `text` y parseo.

Funciones CRUD exportadas

Codigo:

```
export async function getAllCitysStats(query = {}) {
  const response = await fetch(buildUrl("", query));
  return handleResponse(response);
}

export async function createCityStat(cityStat) {
  const response = await fetch(buildUrl(), {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(cityStat)
  });
  return handleResponse(response);
}

export async function updateCityStat(city, country, cityStat) {
  const response = await fetch(
    buildUrl(`/${encodeURIComponent(city)}/${encodeURIComponent(country)}`),
    {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(cityStat)
    }
  );
  return handleResponse(response);
}
```

Que hacen:

Son la puerta de entrada del frontend al CRUD. Cada una representa una accion REST: listar, crear o actualizar.

Como te pueden pedir cambiarlas:

- Endpoint nuevo: crear una funcion nueva aqui y llamarla desde la pantalla.
- Header nuevo: anadirlo en el `fetch` correspondiente.
- Cambiar version API: revisar `CITYS_STATS_API_BASE`.

16.6 Pantallas Svelte principales

`validateCityStatForm`

Archivo:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Codigo:

```
function validateCityStatForm(form) {
  const city = String(form.city ?? "").trim();
  const country = String(form.country ?? "").trim();
```

```

const un_2025_population = parsePositiveInteger(
  form.un_2025_population,
  "Poblacion estimada en 2025"
);

if (!city) {
  throw new Error("Indique una ciudad.");
}

if (!country) {
  throw new Error("Indique un pais.");
}

return { city, country, un_2025_population };
}

```

Que hace:

Valida el formulario antes del `POST`. Mejora experiencia de usuario, pero no sustituye a la validacion del backend.

Como te pueden pedir cambiarla:

- Anadir campo `continent`: leerlo, validar que no este vacio y devolverlo.
- Permitir decimales o cero: tocar `parsePositiveInteger`.
- Cambiar mensaje visible: modificar los `throw new Error`.

`buildSearchQuery`

Codigo:

```

function buildSearchQuery() {
  const query = {
    q: String(searchForm.q ?? "").trim(),
    city: String(searchForm.city ?? "").trim(),
    country: String(searchForm.country ?? "").trim(),
    un_2025_population: parseOptionalPositiveInteger(
      searchForm.un_2025_population,
      "Poblacion exacta"
    ),
    sort: String(searchForm.sort ?? "").trim(),
    limit: parseOptionalNonNegativeInteger(searchForm.limit, "Numero maximo de resultados"),
    offset: parseOptionalNonNegativeInteger(searchForm.offset, "Posicion inicial")
  };

  Object.keys(query).forEach((key) => {
    if (query[key] === "" || query[key] === undefined) {
      delete query[key];
    }
  });

  return query;
}

```

Que hace:

Convierte inputs de búsqueda en query params limpios para `GET /api/v2/citys-stats`.

Como te pueden pedir cambiarla:

- Filtro `min_population`: anadir campo en `emptySearchForm`, input HTML y propiedad aqui.
- Nuevo orden: anadir opcion en `sortOptions`; si es campo nuevo, tambien backend.
- Mandar campos vacios: quitar el `delete`, aunque ahora se evita ruido.

`refreshList`

Codigo:

```
async function refreshList(query = activeQuery, successMessage = "") {
  loading = true;
  error = "";

  try {
    citysStats = await getAllCitysStats(query);
    activeQuery = { ...query };

    if (successMessage) {
      message = successMessage;
    }

    return citysStats;
  } catch (e) {
    citysStats = [];
    error = e.message || "No se pudieron cargar los datos de ciudades.";
    return [];
  } finally {
    loading = false;
  }
}
```

Que hace:

Recarga tabla, mantiene filtros activos y controla estados `loading`, `message` y `error`.

Como te pueden pedir cambiarla:

- Mantener tabla si falla: no vaciar `citysStats` en `catch`.
- Mostrar contador total: usar `citysStats.length` o crear endpoint `/count`.
- Recargar sin filtros despues de crear: llamar con `{}` en vez de `activeQuery`.

`handleCreate`

Codigo:

```
async function handleCreate() {
  clearFeedback();

  try {
    const payload = validateCityStatForm(createForm);
    const created = await createCityStat(payload);
  }
```

```

    createForm = emptyCreateForm();
    await refreshList(
      activeQuery,
      `Se ha creado el registro de ${created.city} (${created.country}).`
    );
  } catch (e) {
    error = e.message || "No se pudo crear el registro.";
  }
}

```

Que hace:

Representa el flujo del boton de crear: limpiar mensajes, validar, llamar service, limpiar formulario y recargar tabla.

Como te pueden pedir cambiarla:

- Redirigir a edicion tras crear: llamar a `openEdit(created.city, created.country)`.
- No limpiar formulario: quitar `createForm = emptyCreateForm()`.
- Mostrar mensaje distinto: cambiar el texto enviado a `refreshList`.

handleUpdate

Archivo:

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

Fragmento clave:

```

async function handleUpdate() {
  clearFeedback();

  try {
    const payload = validateForm();

    if (isSameResource(payload)) {
      const updated = await updateCityStat(
        originalKey.city,
        originalKey.country,
        payload
      );

      form = {
        city: updated.city,
        country: updated.country,
        un_2025_population: updated.un_2025_population
      };

      message = "Los cambios se han guardado correctamente.";
      return;
    }

    const created = await createCityStat(payload);
    await deleteCityStat(originalKey.city, originalKey.country);
  }
}

```

```

    originalKey = { city: created.city, country: created.country };
    updateRoute(created.city, created.country);
  } catch (e) {
    error = e.message || "No se pudieron guardar los cambios.";
  }
}

```

Que hace:

Si la clave `city + country` no cambia, usa `PUT`. Si cambia, crea el nuevo recurso y borra el anterior.

Como te pueden pedir cambiarla:

- Hacer siempre `PUT`: solo si el backend permite cambiar clave en `PUT`.
- Pedir confirmacion antes de borrar: anadir confirmacion antes de `deleteCityStat`.
- Cambiar la ruta tras editar: tocar `updateRoute`.

Respuesta de defensa:

Como `city + country` identifica el recurso, cambiar la clave equivale a moverlo. Por eso el frontend crea el nuevo y borra el antiguo.

16.7 Analytics, mapa e integraciones UI

`renderChart` de analytics individual

Codigo resumido:

```

function renderChart() {
  if (!chartContainer) return;

  const data = citysStats
    .slice()
    .sort((a, b) => Number(b.un_2025_population) - Number(a.un_2025_population))
    .map((item) => ({
      name: labelFor(item),
      y: Number(item.un_2025_population)
    }));

  chart?.destroy();

  chart = Highcharts.chart(chartContainer, {
    chart: { type: "pie", backgroundColor: "transparent" },
    series: [{ name: "Poblacion 2025", colorByPoint: true, data }]
  });
}

```

Que hace:

Prepara datos de poblacion y pinta una grafica Highcharts. Destruye la anterior para no duplicar instancias.

Como te pueden pedir cambiarla:

- Cambiar `pie` a `bar`: cambiar `chart.type`, `xAxis.categories` y forma de `series`.
- Mostrar top 5: aplicar `.slice(0, 5)` despues de ordenar.
- Cambiar metrica: mapear otro campo en `y`.

`renderMap`

Fragmento clave:

```
function renderMap() {
  if (!mapContainer || !Highcharts || points.length === 0) return;

  mapChart?.destroy();

  mapChart = Highcharts.mapChart(mapContainer, {
    chart: { map: worldMap, backgroundColor: "transparent" },
    series: [
      { name: "Países", nullColor: "#dbe6dd" },
      {
        type: "mappoint",
        name: "Ciudades",
        data: points.map((point) => ({
          name: titleCase(point.city),
          lat: point.lat,
          lon: point.lon,
          marker: { radius: point.radius, fillColor: point.color },
          custom: point
        })))
      ]
    ]
  });
}
```

Que hace:

Crea un mapa mundial con marcadores para ciudades geolocalizadas.

Como te pueden pedir cambiarla:

- Anadir ciudad al mapa: tocar `coordinates`, no solo `renderMap`.
- Cambiar color: tocar `colorFor` o `colorAxis`.
- Cambiar tamano: tocar `radiusFor`.
- Cambiar tooltip: tocar `tooltip.pointFormatter`.

`safeLoad`

Archivo:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Codigo:

```

async function safeLoad(task) {
  try {
    return { data: await task(), error: "" };
  } catch (e) {
    return { data: null, error: e.message || "No se pudo cargar la integracion." };
  }
}

```

Que hace:

Evita que una llamada externa fallida rompa toda la pantalla de integraciones.

Como te pueden pedir cambiarla:

- Parar toda la pantalla si falla una API: quitar `safeLoad` y dejar que el error suba al `catch` general.
- Mostrar mas informacion: devolver tambien `source` o `context`.
- Ocultar errores: no llamar a `collectError`, aunque perderias transparencia.

`loadIntegrations`

Fragmento clave:

```

async function loadIntegrations() {
  loading = true;
  error = "";
  integrationErrors = [];
  restoredInitialData = false;
  destroyIntegrationCharts();

  try {
    countrySummaries = await getCountrySummaries(selectedLimit);

    if (countrySummaries.length === 0) {
      await loadInitialCityStats();
      countrySummaries = await getCountrySummaries(selectedLimit);
      restoredInitialData = countrySummaries.length > 0;
    }

    const geocodingResults = await Promise.all(
      countrySummaries.map((item) =>
        safeLoad(() => getGeocoding(item.topCity, item.country))
      )
    );

    const [tourismResult, earthquakeResult, fifaResult, esportsResult] =
      await Promise.all([
        safeLoad(getSosTouristArrivals),
        safeLoad(getSosEarthquakes),
        safeLoad(getSosFifaSquadValues),
        safeLoad(getSosEsportsEarnings)
      ]);

    loading = false;
    await tick();
    await loadHighcharts();
    renderIntegrationCharts();
  }
}

```

```

    } catch (e) {
      error = e.message || "No se pudieron cargar las integraciones.";
      loading = false;
    }
  }
}

```

Que hace:

Orquesta toda la vista de integraciones: carga datos locales, recupera APIs externas en paralelo, registra errores parciales, espera al DOM y pinta widgets.

Como te pueden pedir cambiarla:

- Anadir API externa: importar service, llamar con `safeLoad`, guardar estado, pintar widget y anadirlo a `renderIntegrationCharts`.
- Cambiar limite: tocar `selectedLimit` y el selector HTML.
- Quitar autocarga inicial: eliminar el bloque `if (countrySummaries.length === 0)`.

Respuesta de defensa:

En integraciones hay operaciones independientes. Por eso uso `Promise.all`: no espero una API para empezar la siguiente.

16.8 Flujos completos que conviene memorizar

La chuleta completa esta en la seccion 17, para evitar duplicar informacion.

Idea que hay que memorizar:

El orden real depende del contexto: pantalla, boton o endpoint. Las funciones no se ejecutan por estar arriba o abajo en el archivo, sino porque algo las llama.

16.9 Recetas de cambio que mas pueden pedir

Anadir campo `continent`

Backend v2:

```

const expected = ["city", "country", "continent", "un_2025_population"].sort();

const continent = String(body.continent).trim().toLowerCase();

if (!city || !country || !continent || !Number.isInteger(un_2025_population)) {
  return null;
}

return { city, country, continent, un_2025_population };

```

Tambien revisar:

- `initialData` en v1 y v2.
- `tests/LCC` porque `POST` y `PUT` necesitan el campo nuevo.
- `emptycreateForm`, `emptysearchform`, `validateCityStatForm` y HTML del CRUD.
- Pantalla de edicion: `emptyForm`, `loadResource`, `validateForm`, `handleUpdate`.
- Tablas, analytics, mapa e integraciones si se quiere mostrar o usar `continent`.

Frase de defensa:

Como cambia el contrato del recurso, no basta con anadir un input. Hay que tocar backend, frontend y tests.

Anadir filtro `min_population`

Backend:

```
if (req.query.min_population !== undefined) {
  const value = Number(req.query.min_population);
  if (!Number.isFinite(value) || value < 0) {
    return res.status(400).json({ error: "Invalid query" });
  }
  result = result.filter(d => d.un_2025_population >= value);
}
```

Frontend:

```
min_population: parseOptionalNonNegativeInteger(
  searchForm.min_population,
  "Poblacion minima"
)
```

Tambien revisar:

- `emptySearchForm`.
- Input HTML de busqueda.
- Tests de query valida e invalida.

Anadir endpoint `/api/v2/citys-stats/count`

Poner antes de la ruta parametrizada `/:city/:country`:

```
app.get(`${BASE_API_URL}/count`, (req, res) => {
  db.count({}, (err, count) => {
    if (err) return res.sendStatus(500);
    return res.status(200).json({ count });
  });
});
```

Por que antes:

Express evalúa rutas en orden. Las rutas especiales se colocan antes de las rutas con parámetros para evitar capturas accidentales.

Frontend:

```
export async function getCityStatsCount() {
  const response = await fetch(buildUrl("/count"));
  return handleResponse(response);
}
```

Cambiar gráfica **pie** a **bar**

No basta siempre con cambiar `type`. Para barras suele hacer falta eje X y serie numérica:

```
chart = Highcharts.chart(chartContainer, {
  chart: { type: "bar", backgroundColor: "transparent" },
  xAxis: {
    categories: data.map((item) => item.name)
  },
  series: [
    {
      name: "Población 2025",
      data: data.map((item) => item.y)
    }
  ]
});
```

Revisar tooltip, altura del contenedor y tests visuales si los hay.

Añadir una API externa nueva

Backend:

1. Crear constante de URL externa.
2. Crear función que use `fetchJson`.
3. Normalizar la respuesta si viene con nombres raros.
4. Crear endpoint proxy en `src/back/v1/citys-stats.js`.
5. Si entra en resumen, meterla en `/integrations/summary`.

Service:

```
export async function getNuevaApi() {
  const response = await fetch(`${CITYS_STATS_INTEGRATIONS_API_BASE}/integrations/nueva-api`);
  return handleResponse(response);
}
```

Pantalla:

1. Importar service.
2. Crear estado para sus filas.
3. Llamar con `safeLoad` dentro de `loadIntegrations`.

4. Crear `renderNuevaApiChart`.
5. Anadirlo a `renderIntegrationCharts`.
6. Anadir panel HTML.
7. Destruir la grafica con las demas.

Frase de defensa:

Primero backend proxy, luego service, luego pantalla. Asi el navegador no depende directamente de la API externa.

16.10 Asincronia sin liarse

Reglas simples:

```
callback de NeDB: db.find(..., (err, docs) => { ... })
async/await: espera peticiones HTTP o imports dinamicos
Promise.all: lanza varias llamadas en paralelo y espera todas
tick: espera a que Svelte pinte el DOM antes de usar Highcharts
try/catch: convierte errores en mensajes controlados
finally: ejecuta limpieza aunque haya error
```

Frases utiles:

- `db.find` no devuelve los documentos directamente; los entrega en el callback.
- `await fetch(...)` espera la respuesta de red.
- `Promise.all` acelera integraciones independientes.
- `tick` evita pintar una grafica antes de que exista el contenedor.
- `chart?.destroy()` evita duplicar graficas al repintar.

16.11 Tabla de impacto: si tocas esto, revisa esto

Cambio	Revisar tambien
<code>hasExactCityFields</code>	<code>normalizeCityStat</code> , bodies de tests, formularios Svelte
<code>normalizeCityStat</code>	datos iniciales, mensajes de error, POST, PUT, edicion
Handler <code>GET</code> v2	<code>buildSearchQuery</code> , tests de filtros, analytics y mapa
Handler <code>POST</code> v2	<code>createCityStat</code> , <code>handleCreate</code> , test de duplicado <code>409</code>
Handler <code>PUT</code> v2	<code>updateCityStat</code> , <code>handleUpdate</code> , regla URL/body
Handler <code>DELETE</code> v2	botones de borrado, mensajes, <code>204</code> y tests
<code>buildCityCountrySummaries</code>	integraciones por pais, <code>summary</code> , widgets y tablas
<code>fetchJson</code>	todos los endpoints externos
<code>safeExternal</code>	errores parciales del backend de integraciones
<code>apiPath</code>	todas las llamadas <code>fetch</code> del frontend
<code>handleResponse</code>	todos los mensajes de error visibles

Cambio	Revisar tambien
<code>refreshList</code>	listado, busqueda, crear, borrar y cargar iniciales
<code>handleUpdate</code>	pantalla de edicion y regla de clave compuesta
<code>renderChart</code>	grafica individual, tooltip, <code>onDestroy</code>
<code>renderMap</code>	coordenadas, colores, radios, tooltip
<code>loadIntegrations</code>	todas las APIs externas y widgets Highcharts

16.12 Preguntas dificiles sobre funciones

Pregunta	Respuesta segura
Por que quitas <code>_id</code> ?	Porque es interno de NeDB y no pertenece al contrato publico.
Por que validas en frontend y backend?	Frontend mejora UX; backend protege el sistema real.
Por que <code>POST</code> devuelve <code>409</code> ?	Porque ya existe un recurso con la misma clave <code>city + country</code> .
Por que <code>PUT</code> compara URL y body?	Para asegurar que actualizo exactamente el recurso identificado por la URL.
Por que integras por pais?	REST Countries, World Bank y varias APIs SOS encajan mejor por <code>country</code> que por ciudad.
Por que <code>safeLoad</code> no lanza error?	Porque en integraciones interesa mostrar datos parciales aunque una fuente falle.
Por que <code>Promise.all</code> ?	Para cargar APIs independientes en paralelo y reducir espera.
Por que <code>tick</code> antes de Highcharts?	Porque Highcharts necesita que el contenedor ya exista en el DOM.
Donde cambiarias mensajes de error?	En <code>friendlyApiMessage</code> para CRUD y en <code>handleResponse</code> de integraciones si son externas.
Como buscas quien llama a una funcion?	Con <code>rg "nombreFuncion"</code> desde la raiz del repositorio.

16.13 Mini guion si te piden modificar algo en directo

1. Identificar capa: API, service, pantalla, grafica o test.
2. Buscar funcion con `rg` .
3. Hacer cambio minimo.
4. Revisar funciones que la llaman.
5. Ejecutar test o comprobacion manual.
6. Explicar que se ha cambiado y por que.

Comandos utiles:

```
rg "normalizeCityStat"
rg "buildSearchQuery"
rg "loadIntegrations"
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Regla final:

Si cambia el contrato de datos, toca backend, frontend y tests. Si cambia solo la visualizacion, normalmente toca Svelte y quiza el service si necesita datos nuevos.

17. Flujos de ejecucion

Idea clave para defensa:

El orden real no depende de donde este escrita la funcion, sino de quien la llama: una ruta de Express, un `onMount`, un boton o un service del frontend.

Otra idea importante:

Cuando pongo "No hay funcion auxiliar", significa que ese paso esta escrito directamente dentro de la ruta o del componente. No te falta memorizar ninguna funcion con nombre.

Arranque general

1. El usuario ejecuta `npm start`.
2. Node ejecuta `index.js`.
3. Express crea `app`.
4. Se activan CORS y lectura JSON.
5. Se abren las bases NeDB.
6. Se importan APIs v1 y v2.
7. Cada API ejecuta `module.exports(app, db)`.
8. Cada API crea constantes, declara funciones y registra rutas.
9. Se sirve `public`.
10. Se registra el fallback de la SPA.
11. El servidor queda escuchando.

Importante:

- En el arranque las funciones auxiliares se declaran, pero no se ejecutan.
- Las rutas `app.get`, `app.post`, `app.put` y `app.delete` solo se registran.
- Una ruta se ejecuta solo cuando llega una peticion con su metodo y URL.

Regla para cualquier pantalla Svelte

1. El usuario abre una URL, por ejemplo `/citys-stats`.
2. Express devuelve `public/index.html`.
3. Arranca Svelte.
4. `App.svelte` mira `window.location.pathname`.

5. El router carga el componente de esa ruta.
6. El `<script>` del componente se ejecuta.
7. Las funciones se declaran.
8. `onMount` ejecuta la carga inicial si existe.
9. Los botones ejecutan sus handlers solo cuando el usuario pulsa.

Si estas en `/citys-stats`

Abrir la pantalla:

```
onMount
-> refreshList({}, "")
-> getAllCitysStats({})
-> buildUrl("", {})
-> fetch GET /api/v2/citys-stats
-> backend v2 app.get(BASE_API_URL)
-> db.find
-> docs.map(removeDatabaseId)
-> filtros exactos si vienen
    No hay funcion auxiliar: se hace dentro de la ruta.
-> busqueda q si viene
    No hay funcion auxiliar: se hace dentro de la ruta.
-> sort si viene
    No hay funcion auxiliar: se hace dentro de la ruta.
-> offset/limit si vienen
    No hay funcion auxiliar: se hace dentro de la ruta.
-> result.slice(offset, offset + limit)
-> res.status(200).json(...)
-> handleResponse(response)
-> response.text()
-> JSON.parse si hay cuerpo
-> citysStats = datos recibidos
-> activeQuery = {}
-> loading = false
```

Buscar:

```
handleSearch
-> clearFeedback
-> buildSearchQuery
-> parseOptionalPositiveInteger para un_2025_population
-> parseOptionalNonNegativeInteger para limit
-> parseOptionalNonNegativeInteger para offset
-> refreshList(query)
-> getAllCitysStats(query)
-> buildUrl("", query)
-> fetch GET /api/v2/citys-stats?...query
-> backend v2 app.get(BASE_API_URL)
-> db.find
-> docs.map(removeDatabaseId)
-> filtro city si viene
    No hay funcion auxiliar.
-> filtro country si viene
    No hay funcion auxiliar.
```

```

        -> filtro un_2025_population si viene
            No hay funcion auxiliar.
        -> busqueda q si viene
            No hay funcion auxiliar.
        -> sort si viene
            No hay funcion auxiliar.
        -> offset/limit si vienen
            No hay funcion auxiliar.
        -> result.slice(...)
        -> res.status(200).json(...)
    -> handleResponse(response)
-> hasQueryValues(query)
-> message o error

```

Limpiar filtros:

```

handleResetSearch
-> emptySearchForm
-> clearFeedback
-> refreshList({})
    -> getAllCityStats({})
        -> buildUrl("", {})
        -> fetch GET /api/v2/citys-stats
        -> backend v2 app.get(BASE_API_URL)
        -> handleResponse(response)

```

Crear:

```

handleCreate
-> clearFeedback
-> validateCityStatForm
    -> parsePositiveInteger
-> createCityStat
    -> buildUrl()
    -> fetch POST /api/v2/citys-stats
        -> backend v2 app.post(BASE_API_URL)
            -> normalizeCityStat
                -> hasExactCityFields
            -> db.findOne para duplicado
            -> db.insert
            -> removeDatabaseId
            -> res.status(201).json(...)
    -> handleResponse(response)
-> emptyCreateForm
-> refreshList(activeQuery)
    -> getAllCityStats(activeQuery)
    -> buildUrl("", activeQuery)
    -> fetch GET /api/v2/citys-stats
    -> backend v2 app.get(BASE_API_URL)
    -> handleResponse(response)

```

Cargar datos iniciales:

```

handleLoadInitialData
-> clearFeedback

```

```

-> loadInitialCityStats
-> buildUrl("/loadInitialData")
-> fetch GET /api/v2/citys-stats/loadInitialData
  -> backend v2 app.get(`${BASE_API_URL}/loadInitialData`)
    -> db.count
    -> si count > 0:
      -> db.find
      -> docs.map(removeDatabaseId)
      -> res.status(200).json(...)
    -> si count === 0:
      -> db.insert(initialData)
      -> docs.map(removeDatabaseId)
      -> res.status(201).json(...)
  -> handleResponse(response)
-> refreshList(activeQuery)

```

Borrar uno:

```

handleDeleteOne
-> clearFeedback
-> deleteCityStat
  -> encodePathValue(city)
  -> encodePathValue(country)
  -> buildUrl("/:city/:country")
  -> fetch DELETE /api/v2/citys-stats/:city/:country
    -> backend v2 app.delete(`${BASE_API_URL}/:city/:country`)
      -> normaliza req.params.city y req.params.country
      -> db.remove({ city, country }, {})
      -> res.sendStatus(204)
      -> o res.status(404).json(...) si no existia
    -> handleResponse(response)
-> refreshList(activeQuery)

```

Borrar todos:

```

handleDeleteAll
-> clearFeedback
-> deleteAllCityStats
  -> buildUrl()
  -> fetch DELETE /api/v2/citys-stats
    -> backend v2 app.delete(BASE_API_URL)
      -> db.remove({}, { multi: true })
      -> res.sendStatus(204)
    -> handleResponse(response)
-> refreshList(activeQuery)

```

Abrir edicion:

```

openEdit
-> encodeURIComponent(city)
-> encodeURIComponent(country)
-> navigate("/citys-stats/editar/:city/:country")

```

Si estas en `/citys-stats/editar/:city/:country`

Abrir pantalla:

```
onMount
  -> loadResource
    -> getOneCityStat(params.city, params.country)
    -> encodePathValue(city)
    -> encodePathValue(country)
    -> buildUrl("/:city/:country")
    -> fetch GET /api/v2/citys-stats/:city/:country
      -> backend v2 app.get(`${BASE_API_URL}/${city}/${country}`)
      -> normaliza req.params.city y req.params.country
      -> db.findOne({ city, country })
      -> removeDatabaseId
      -> res.status(200).json(...)
      -> o 404 si no existe
    -> handleResponse(response)
  -> rellena form
  -> rellena originalKey
```

Guardar sin cambiar `city` ni `country`:

```
handleUpdate
  -> clearFeedback
  -> validateForm
    -> parsePositiveInteger
  -> isSameResource true
  -> updateCityStat
    -> encodePathValue(originalKey.city)
    -> encodePathValue(originalKey.country)
    -> buildUrl("/:city/:country")
    -> fetch PUT /api/v2/citys-stats/:city/:country
      -> backend v2 app.put(`${BASE_API_URL}/${city}/${country}`)
      -> normalizeCityStat
        -> hasExactCityFields
      -> comprueba URL y body
      -> db.findOne
      -> db.update
      -> db.findOne
      -> removeDatabaseId
      -> res.status(200).json(...)
    -> handleResponse(response)
  -> actualiza form
  -> message
```

Guardar cambiando `city` o `country`:

```
handleUpdate
  -> clearFeedback
  -> validateForm
    -> parsePositiveInteger
  -> isSameResource false
  -> createCityStat(payload)
    -> buildUrl()
    -> fetch POST /api/v2/citys-stats
```

```

-> backend v2 app.post(BASE_API_URL)
  -> normalizeCityStat
    -> hasExactCityFields
  -> db.findOne
  -> db.insert
  -> removeDatabaseId
-> handleResponse(response)
-> deleteCityStat(originalKey.city, originalKey.country)
  -> encodePathValue
  -> buildUrl("/:city/:country")
  -> fetch DELETE /api/v2/citys-stats/:city/:country
  -> backend v2 app.delete(`${BASE_API_URL}/${city}/${country}`)
    -> db.remove
  -> handleResponse(response)
-> updateRoute(created.city, created.country)
  -> encodeURIComponent
  -> replace("/citys-stats/edtar/:city/:country")

```

Si estas en `/analytics/citys-stats`

```

onMount
-> loadAnalytics
  -> getAllCityStats({ sort: "-un_2025_population" })
  -> buildUrl("", { sort })
  -> fetch GET /api/v2/citys-stats?sort=-un_2025_population
    -> backend v2 app.get(BASE_API_URL)
      -> db.find
      -> docs.map(removeDatabaseId)
      -> sort inline dentro de la ruta
      -> result.slice(...)
    -> handleResponse(response)
-> tick
-> loadHighcharts
  -> import("highcharts")
  -> import("highcharts/modules/accessibility.js")
-> renderChart
  -> citysStats.slice()
  -> sort por poblacion
  -> map(...)
    -> labelFor(item)
  -> chart?.destroy()
  -> Highcharts.chart(...)
onDestroy
-> chart?.destroy()

```

Si estas en `/analytics/citys-stats/map`

```

onMount
-> loadMapData
  -> getAllCityStats({ sort: "-un_2025_population" })
  -> buildUrl("", { sort })
  -> fetch GET /api/v2/citys-stats?sort=-un_2025_population
  -> backend v2 app.get(BASE_API_URL)
  -> handleResponse(response)
-> declaraciones reactivas de Svelte:
  -> geolocated = citysStats.map(...)

```

```

        -> keyFor(item)
    -> missing = citysStats.filter(...)
        -> keyFor(item)
    -> maxPopulation = Math.max(...)
    -> minPopulation = Math.min(...)
    -> points = geolocated.map(...)
        -> colorFor(item.population)
        -> radiusFor(item.population)
    -> selectedPoint = points.find(...)
    -> totalPopulation = geolocated.reduce(...)
-> tick
-> loadHighchartsMap
    -> import("highcharts")
    -> import("highcharts/modules/map.js")
    -> import("highcharts/modules/accessibility.js")
-> renderMap
    -> points.map(...)
        -> titleCase(point.city)
        -> titleCase(point.country)
    -> Highcharts.mapChart(...)
    -> evento render
        -> removeCityMarkerClip(chart)
    -> eventos click/mouseOver
        -> selectPoint(point)
onDestroy
    -> mapChart?.destroy()

```

Si estas en `/integrations/citys-stats`

La pantalla se puede lanzar de tres formas:

```

onMount(loadIntegrations)
select on:change={loadIntegrations}
button on:click={loadIntegrations}

```

Carga local inicial:

```

loadIntegrations
  -> destroyIntegrationCharts
  -> getCountrySummaries(selectedLimit)
    -> fetch GET /api/v1/citys-stats/country-summaries?limit=N
    -> backend v1 app.get(`${BASE_API_URL}/country-summaries`)
        -> parseLimit
        -> findAllCityStats
            -> db.find
            -> docs.map(removeDatabaseId)
        -> buildCityCountrySummaries
            -> normalizeCountryKey
            -> readFiniteNumber
        -> slice(0, limit)
        -> res.status(200).json(...)
    -> handleResponse(response)

```

Si no hay datos locales:

```

if countrySummaries.length === 0
  -> loadInitialCityStats
  -> buildUrl("/loadInitialData")
  -> fetch GET /api/v2/citys-stats/loadInitialData
  -> backend v2 app.get(`${BASE_API_URL}/loadInitialData`)
  -> db.count
  -> db.find o db.insert(initialData)
  -> removeDatabaseId
  -> handleResponse(response)
  -> getCountrySummaries(selectedLimit) otra vez

```

Bloque Open-Meteo:

```

Promise.all(countrySummaries.map(...))
  -> safeLoad(() => getGeocoding(item.topCity, item.country))
  -> getGeocoding del service
  -> fetch GET /api/v1/citys-stats/integrations/geocoding/:city
  -> backend v1 app.get(`${BASE_API_URL}/integrations/geocoding/:city`)
  -> getGeocoding del backend
  -> cleanSearchTerm(city)
  -> cleanSearchTerm(country)
  -> fetchJson(Open-Meteo)
  -> fetch
  -> response.text()
  -> JSON.parse
  -> res.status(200).json(...)
  -> handleResponse(response)
-> geocodingResults.map(...)
  -> collectError
  -> titleCase en el contexto del error
-> geocodingRows

```

Bloque REST Countries:

```

Promise.all(countrySummaries.map(...))
  -> safeLoad(() => getCountryInfo(item.country))
  -> getCountryInfo del service
  -> fetch GET /api/v1/citys-stats/integrations/country/:country
  -> backend v1 app.get(`${BASE_API_URL}/integrations/country/:country`)
  -> getCountryInfo del backend
  -> cleanSearchTerm(country)
  -> fetchJson(REST Countries)
  -> selecciona pais correcto
  -> res.status(200).json(...)
  -> handleResponse(response)
-> countryResults.map(...)
  -> collectError
  -> titleCase
-> countryCards

```

Bloque World Bank:

```

Promise.all(countryCards.map(...))
  -> si no hay cca3:
  -> Promise.resolve({ data: null, error: "Codigo ISO3 no disponible" })

```

```

-> si hay cca3:
  -> safeLoad(() => getWorldBankPopulation(row.countryData.cca3))
  -> getWorldBankPopulation del service
    -> fetch GET /api/v1/citys-stats/integrations/world-bank/:countryCode
      -> backend v1 app.get(`${BASE_API_URL}/integrations/world-bank/:countryCode`)
        -> getWorldBankPopulation del backend
          -> mira worldBankPopulationCache
          -> fetchJson(World Bank) si no esta en cache
          -> normalizeWorldBankRow
          -> guarda en cache
        -> res.status(200).json(...)
      -> handleResponse(response)
-> worldBankResults.map(...)
  -> collectError
  -> titleCase
-> worldBankRows

```

Bloque APIs SOS externas:

```

Promise.all([
  safeLoad(getSosTouristArrivals),
  safeLoad(getSosEarthquakes),
  safeLoad(getSosFifaSquadValues),
  safeLoad(getSosEsportsEarnings)
])

getSosTouristArrivals
  -> fetch GET /api/v1/citys-stats/integrations/sos-tourist-arrivals
    -> backend getTouristArrivals
      -> fetchJson(SOS2526-25)
      -> asArray
      -> normalizeTouristArrival
        -> readFiniteNumber
        -> filter(Boolean)
      -> buildTouristArrivalsByCountry
        -> normalizeCountryKey
      -> sort por totalArrivals
    -> handleResponse

getSosEarthquakes
  -> fetch GET /api/v1/citys-stats/integrations/sos-earthquakes
    -> backend getEarthquakes
      -> fetchJson(SOS2526-19)
      -> asArray
      -> normalizeEarthquake
        -> countryFromIso3
        -> readFiniteNumber
        -> filter(Boolean)
      -> buildEarthquakesByCountry
        -> normalizeCountryKey
      -> sort por maxSeverity
    -> handleResponse

getSosFifaSquadValues
  -> fetch GET /api/v1/citys-stats/integrations/sos-fifa-squad-values
    -> backend getFifaSquadValues
      -> fetchJson(SOS2526-26)
      -> asArray

```

```

    -> normalizeFifaSquadValue
    -> readFiniteNumber
    -> filter(Boolean)
  -> buildFifaSquadValuesByCountry
    -> normalizeCountryKey
  -> sort por latestTotalMarketValue
-> handleResponse

getSosEsportsEarnings
  -> fetch GET /api/v1/citys-stats/integrations/sos-esports-earnings
  -> backend getEsportsEarnings
    -> fetchJson(SOS2526-30)
    -> asArray
    -> normalizeEsportsEarning
      -> readFiniteNumber
    -> filter(Boolean)
  -> buildEsportsEarningsByCountry
    -> normalizeCountryKey
  -> sort por topCountryEarnings
-> handleResponse

```

Preparar datos finales de pantalla:

```

collectError para turismo, terremotos, FIFA y eSports
touristCountries = topCountries(tourismResult.data, "totalArrivals")
  -> numberOrNull
earthquakeCountries = topCountries(earthquakeResult.data, "maxSeverity")
  -> numberOrNull
fifaCountries = topCountries(fifaResult.data, "latestTotalMarketValue")
  -> numberOrNull
esportsCountries = topCountries(esportsResult.data, "topCountryEarnings")
  -> numberOrNull
integrationErrors = errors

```

Final de la pantalla:

```

loading = false
tick
loadHighcharts
  -> import("highcharts")
  -> import("highcharts/highcharts-more.js")
  -> loadPlugin(dumbbell)
  -> loadPlugin(lollipop)
  -> loadPlugin(variwide)
  -> loadPlugin(bullet)
  -> loadPlugin(sankey)
  -> loadPlugin(treemap)
  -> loadPlugin(sunburst)
  -> loadPlugin(accessibility)
renderIntegrationCharts
  -> destroyIntegrationCharts
  -> renderGeocodingChart
    -> numberOrNull
    -> titleCase
    -> displayNumber/displayDecimal
    -> createChart
  -> renderCountryChart

```

```

    -> numberOrNull
    -> countryName
    -> displayNumber/displayCompact
    -> createChart
-> renderWorldBankChart
    -> numberOrNull
    -> countryName
    -> displayNumber
    -> createChart
-> renderTourismChart
    -> combinedCountryRows
        -> findLocalCountry
            -> localCountryIndex
                -> normalizeCountryKey
        -> localAveragePopulation si no hay match
    -> localPopulationFor
    -> titleCase
    -> createChart
-> renderEarthquakeChart
    -> combinedCountryRows
    -> localPopulationFor
    -> normalizedIndex
        -> numberOrNull
    -> titleCase
    -> createChart
-> renderFifaChart
    -> numberOrNull
    -> normalizedIndex
    -> titleCase
    -> createChart
-> renderEsportsChart
    -> combinedCountryRows
    -> localPopulationFor
    -> normalizedIndex
    -> titleCase
    -> createChart
onDestroy
    -> destroyIntegrationCharts

```

Si estas en `/analytics`

```

onMount
-> loadAnalytics
    -> Promise.all([
        getAllCityStats(),
        getDisasters(),
        getAllWineStats()
    ])
    -> getAllCityStats
        -> buildUrl
        -> fetch GET /api/v2/citys-stats
        -> backend v2 app.get(BASE_API_URL)
        -> handleResponse
    -> getDisasters
        -> fetch a la API de natural-disasters
    -> getAllWineStats
        -> fetch a la API de wine-stats
    -> buildMetrics(cityStats, disasters, wines)

```

```

    -> sum(cityStats, "un_2025_population")
    -> sum(disasters, "death_count")
    -> sum(wines, "unit")
    -> calcula index normalizado
  -> tick
  -> loadHighcharts
    -> import("highcharts")
    -> import("highcharts/modules/accessibility.js")
  -> renderChart
    -> Highcharts.chart(...)
onDestroy
  -> chart?.destroy()

```

Si llamas directamente a la API v2

GET /api/v2/citys-stats :

```

app.get(BASE_API_URL)
  -> db.find
  -> docs.map(removeDatabaseId)
  -> filtro city si viene
    No hay funcion auxiliar.
  -> filtro country si viene
    No hay funcion auxiliar.
  -> filtro un_2025_population si viene
    No hay funcion auxiliar.
  -> busqueda q si viene
    No hay funcion auxiliar.
  -> sort si viene
    No hay funcion auxiliar.
  -> offset/limit si vienen
    No hay funcion auxiliar.
  -> result.slice(offset, offset + limit)
  -> res.status(200).json(...)

```

POST /api/v2/citys-stats :

```

app.post(BASE_API_URL)
  -> normalizeCityStat
    -> hasExactCityFields
  -> db.findOne duplicado
  -> db.insert
  -> removeDatabaseId
  -> res.status(201).json(...)

```

PUT /api/v2/citys-stats/:city/:country :

```

app.put(`${BASE_API_URL}/:city/:country`)
  -> normalizeCityStat
    -> hasExactCityFields
  -> comprobar URL/body
  -> db.findOne
  -> db.update
  -> db.findOne

```

```
-> removeDatabaseId
-> res.status(200).json(...)
```

DELETE /api/v2/citys-stats:

```
app.delete(BASE_API_URL)
-> db.remove({}, { multi: true })
-> res.sendStatus(204)
```

DELETE /api/v2/citys-stats/:city/:country:

```
app.delete(`${BASE_API_URL}/${city}/${country}`)
-> normaliza req.params.city y req.params.country
-> db.remove({ city, country }, {})
-> res.sendStatus(204)
-> o res.status(404).json(...) si no existia
```

Si llamas directamente a integraciones v1

Endpoints individuales:

```
GET /api/v1/citys-stats/integrations/geocoding/:city
-> getGeocoding
-> cleanSearchTerm
-> fetchJson Open-Meteo
-> fetch
-> response.text()
-> JSON.parse

GET /api/v1/citys-stats/integrations/country/:country
-> getCountryInfo
-> cleanSearchTerm
-> fetchJson REST Countries
-> fetch
-> response.text()
-> JSON.parse

GET /api/v1/citys-stats/integrations/world-bank/:countryCode
-> getWorldBankPopulation
-> worldBankPopulationCache.has
-> fetchJson World Bank si no esta en cache
-> normalizeWorldBankRow
-> worldBankPopulationCache.set

GET /api/v1/citys-stats/integrations/sos-tourist-arrivals
-> getTouristArrivals
-> fetchJson
-> asArray
-> normalizeTouristArrival
-> readFiniteNumber
-> buildTouristArrivalsByCountry
-> normalizeCountryKey
-> sort por totalArrivals

GET /api/v1/citys-stats/integrations/sos-earthquakes
```

```

-> getEarthquakes
  -> fetchJson
  -> asArray
  -> normalizeEarthquake
    -> countryFromIso3
    -> readFiniteNumber
-> buildEarthquakesByCountry
  -> normalizeCountryKey
-> sort por maxSeverity

GET /api/v1/citys-stats/integrations/sos-fifa-squad-values
-> getFifaSquadValues
  -> fetchJson
  -> asArray
  -> normalizeFifaSquadValue
    -> readFiniteNumber
-> buildFifaSquadValuesByCountry
  -> normalizeCountryKey
-> sort por latestTotalMarketValue

GET /api/v1/citys-stats/integrations/sos-esports-earnings
-> getEsportsEarnings
  -> fetchJson
  -> asArray
  -> normalizeEsportsEarning
    -> readFiniteNumber
-> buildEsportsEarningsByCountry
  -> normalizeCountryKey
-> sort por topCountryEarnings

```

Resumen integrado:

```

GET /api/v1/citys-stats/integrations/summary
-> parseLimit
-> findAllCityStats
  -> db.find
  -> docs.map(removeDatabaseId)
-> buildCityCountrySummaries
  -> normalizeCountryKey
  -> readFiniteNumber
-> Promise.all(countrySummaries.map(buildIntegratedCityBase))
  -> buildIntegratedCityBase
    -> Promise.all
      -> safeExternal("Open-Meteo", () => getGeocoding)
        -> getGeocoding
          -> cleanSearchTerm
          -> fetchJson
      -> safeExternal("REST Countries", () => getCountryInfo)
        -> getCountryInfo
          -> cleanSearchTerm
          -> fetchJson
-> extrae countryCodes
-> getWorldBankPopulations
  -> usa worldBankPopulationCache
  -> fetchJson si faltan codigos
  -> normalizeWorldBankRow
-> Promise.all APIs SOS
  -> safeExternal(..., getTouristArrivals)

```

```

-> fetchJson
-> asArray
-> normalizeTouristArrival
-> safeExternal(..., getEarthquakes)
-> fetchJson
-> asArray
-> normalizeEarthquake
-> safeExternal(..., getFifaSquadValues)
-> fetchJson
-> asArray
-> normalizeFifaSquadValue
-> safeExternal(..., getEsportsEarnings)
-> fetchJson
-> asArray
-> normalizeEsportsEarning
-> buildTouristArrivalsByCountry si hay datos
-> buildEarthquakesByCountry si hay datos
-> buildFifaSquadValuesByCountry si hay datos
-> buildEsportsEarningsByCountry si hay datos
-> integrationBases.map(buildIntegratedCity)
  -> normalizeCountryKey
  -> busca datos en Maps externos
  -> crea integrationErrors
-> res.status(200).json(...)

```

Frase corta para examen

Si me dan una funcion, primero miro en que pantalla, boton o endpoint se usa. El archivo se carga de arriba abajo, pero la ejecucion real empieza cuando una ruta de Express, un `onMount` de Svelte o un evento de usuario llama a esa funcion. Si un paso dice "No hay funcion auxiliar", es codigo escrito directamente dentro de esa ruta.

18. Codigos HTTP y contrato REST

Codigo	Uso en este proyecto
200	Lectura correcta o actualizacion con respuesta
201	Recurso creado o datos iniciales insertados
204	Borrado correcto sin cuerpo, usado en eliminaciones correctas
400	Body incorrecto, query invalida o URL/body no coinciden
404	Recurso concreto no encontrado
405	Metodo no permitido para esa URL
409	Duplicado o coleccion ya cargada segun recurso
500	Error interno de servidor o base de datos
502	Error controlado al consultar API externa

Frases utiles:

El código HTTP forma parte del contrato de la API, no es un detalle decorativo.

No usamos URLs tipo `/crearCiudad` porque el método HTTP ya expresa la acción.

`POST` crea en la colección; `PUT` actualiza un recurso que ya existe.

Contrato real de `citys-stats v2`

Esta es la tabla que conviene saberse para defensa LCC. Es la API principal del CRUD.

Método	Ruta	Que hace	Respuestas esperadas
GET	<code>/api/v2/citys-stats/docs</code>	Redirige a documentación Postman	<code>302</code> redirección
GET	<code>/api/v2/citys-stats/loadInitialData</code>	Carga datos iniciales si la base está vacía	<code>201</code> si inserta, <code>200</code> si ya había datos, <code>500</code> si falla NeDB
GET	<code>/api/v2/citys-stats</code>	Lista colección con filtros, <code>q</code> , <code>sort</code> , <code>offset</code> , <code>limit</code>	<code>200</code> , <code>400</code> si query/sort/paginación inválida, <code>500</code>
GET	<code>/api/v2/citys-stats/:city/:country</code>	Devuelve un registro concreto	<code>200</code> , <code>404</code> si no existe, <code>500</code>
POST	<code>/api/v2/citys-stats</code>	Crea un registro nuevo	<code>201</code> , <code>400</code> body inválido, <code>409</code> duplicado, <code>500</code>
POST	<code>/api/v2/citys-stats/:city/:country</code>	No permitido sobre recurso concreto	<code>405</code>
PUT	<code>/api/v2/citys-stats</code>	No permitido sobre colección completa	<code>405</code>
PUT	<code>/api/v2/citys-stats/:city/:country</code>	Actualiza un registro concreto	<code>200</code> , <code>400</code> body inválido, <code>400</code> si URL/body no coinciden, <code>404</code> , <code>500</code>
DELETE	<code>/api/v2/citys-stats</code>	Borra toda la colección	<code>204</code> , <code>500</code>
DELETE	<code>/api/v2/citys-stats/:city/:country</code>	Borra un registro concreto	<code>204</code> , <code>404</code> , <code>500</code>

Frase perfecta:

En REST, la ruta identifica el recurso y el método identifica la acción. Por eso `GET /api/v2/citys-stats` lee la colección, `POST /api/v2/citys-stats` crea en la colección, `PUT /api/v2/citys-stats/:city/:country` actualiza un registro concreto y `DELETE` borra.

Orden interno del `GET /api/v2/citys-stats`

El handler de listado aplica las operaciones en este orden:

1. `db.find({})` lee todos los documentos.
2. `removeDatabaseId` quita `_id` de NeDB.

3. Filtro exacto por city.
4. Filtro exacto por country.
5. Filtro exacto por un_2025_population.
6. Búsqueda libre q en city o country.
7. sort por city, country o un_2025_population.
8. offset y limit para paginacion.
9. res.status(200).json(...)

Por que ese orden:

Primero se obtiene una coleccion limpia, despues se reduce con filtros y busqueda, luego se ordena y al final se pagina. Si paginara antes de filtrar, podria perder resultados correctos.

Ejemplos:

```
GET /api/v2/citys-stats
GET /api/v2/citys-stats?country=china
GET /api/v2/citys-stats?q=india
GET /api/v2/citys-stats?sort=-un_2025_population&limit=5
GET /api/v2/citys-stats?offset=5&limit=5
```

Diferencia entre POST y PUT

POST:

```
POST /api/v2/citys-stats
```

Se usa sobre la coleccion porque el recurso todavia no existe. El backend decide si se puede crear.

Orden:

1. normalizeCityStat(req.body)
2. Si body invalido -> 400
3. db.findOne busca duplicado por city + country
4. Si ya existe -> 409
5. db.insert crea
6. Devuelve 201 con el recurso creado

PUT:

```
PUT /api/v2/citys-stats/:city/:country
```

Se usa sobre un recurso concreto porque el recurso ya existe o deberia existir.

Orden:

1. Lee city y country de req.params.
2. normalizeCityStat(req.body).
3. Si body invalido -> 400.
4. Si body.city/body.country no coinciden con URL -> 400.

```
5. db.findOne comprueba si existe.  
6. Si no existe -> 404.  
7. db.update actualiza.  
8. db.findOne vuelve a leer actualizado.  
9. Devuelve 200 con el recurso actualizado.
```

Frase perfecta:

`POST` no necesita identificador en la URL porque esta creando dentro de la coleccion. `PUT` si lo necesita porque modifica un recurso concreto y por eso compruebo que URL y body coincidan.

Codigos que mas preguntan

`400 Bad Request` :

El cliente ha mandado mal la peticion: body con estructura incorrecta, query invalida, sort no permitido, offset/limit invalidos o URL/body que no coinciden.

`404 Not Found` :

La ruta existe, pero el recurso concreto no esta en la base de datos.

`405 Method Not Allowed` :

La URL existe pero ese metodo no tiene sentido ahi. Por ejemplo, `PUT /api/v2/citys-stats` no se permite porque `PUT` debe ir contra un recurso concreto.

`409 Conflict` :

La peticion esta bien formada, pero choca con el estado actual. En `POST`, ocurre si ya existe la misma combinacion `city + country`.

`500 Internal Server Error` :

Error interno de servidor o base de datos. No es culpa del cliente.

`502 Bad Gateway` :

Se usa en integraciones cuando nuestro backend actua como proxy y falla una API externa.

Contrato de integraciones LCC

Las integraciones estan bajo:

```
/api/v1/citys-stats/integrations/...
```

Endpoints principales:

Ruta	Fuente	Codigos habituales
<code>/integrations/geocoding/:city</code>	Open-Meteo	200 , 404 , 502
<code>/integrations/country/:country</code>	REST Countries	200 , 404 , 502
<code>/integrations/world-bank/:countryCode</code>	World Bank	200 , 404 , 502
<code>/integrations/sos-tourist-arrivals</code>	API SOS externa	200 , 502
<code>/integrations/sos-earthquakes</code>	API SOS externa	200 , 502
<code>/integrations/sos-fifa-squad-values</code>	API SOS externa	200 , 502
<code>/integrations/sos-esports-earnings</code>	API SOS externa	200 , 502
<code>/integrations/summary?limit=5</code>	Resumen combinado	200 , 400 , 500

Frase de defensa:

En integraciones mi backend no es la fuente original de todos los datos, sino un proxy controlador. Por eso si una API externa falla, respondo con errores controlados como 502 o con errores parciales dentro del resumen.

19. Cambios que pueden pedir en directo

Cambiar puerto

Archivo:

```
index.js
```

Pasos:

1. Buscar `const port = process.env.PORT || 10000`.
2. Cambiar `10000` si se pide otro puerto local.
3. Arrancar con `npm.cmd start`.
4. Ver consola.

Frase:

En despliegue manda `process.env.PORT`; el `10000` es el valor por defecto local.

Cambiar donde se guarda una base

Archivo:

```
index.js
```

Buscar:

- `citysStatsDb`

Cambiar:

```
filename: path.join(__dirname, "src", "back", "citys-stats.db")
```

Cambiar datos iniciales

Archivos LCC:

```
src/back/v1/citys-stats.js  
src/back/v2/citys-stats.js
```

Pasos generales:

1. Cambiar `initialData`.
2. Respetar estructura exacta.
3. Borrar coleccion.
4. Cargar datos iniciales.
5. Ejecutar test del recurso.

Ciudades:

```
Invoke-RestMethod -Method Delete http://localhost:10000/api/v2/citys-stats  
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats/loadInitialData
```

Anadir campo a `citys-stats`

Tocar:

```
src/back/v1/citys-stats.js  
src/back/v2/citys-stats.js  
frontend-group/src/routes/citys-stats/+page.svelte  
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte  
tests/LCC/pruebas-lcc*.json  
tests/LCC/e2e/citys-stats.spec.js
```

Pasos:

1. Anadir campo en `hasExactCityFields`.
2. Limpiar y validar en `normalizeCityStat`.
3. Anadirlo a `initialData`.
4. Anadir input en formulario de crear.

5. Anadir input en formulario de editar.
6. Anadir columna en tabla.
7. Revisar analytics si usa el campo.
8. Actualizar tests.

Frase:

Como la API valida estructura exacta, no basta con anadir un input. Hay que aceptar el campo en backend, normalizarlo, mostrarlo y probarlo.

Renombrar campo

Ejemplo:

```
un_2025_population -> population_2025
```

Pasos:

1. Ejecutar `rg "un_2025_population"`.
2. Cambiar backend v1 y v2.
3. Cambiar `initialData`.
4. Cambiar services si construyen payloads.
5. Cambiar formularios y tablas.
6. Cambiar analytics, mapa e integraciones.
7. Cambiar tests.
8. Ejecutar `rg "un_2025_population"` otra vez.

Anadir filtro

Ciudades:

1. Abrir `src/back/v2/citys-stats.js`.
2. Tocar `app.get(BASE_API_URL, ...)`.
3. Validar query param.
4. Abrir `frontend-group/src/routes/citys-stats/+page.svelte`.
5. Anadir campo a `emptySearchForm`.
6. Anadir input.
7. Anadirlo en `buildSearchQuery`.
8. Probar URL.

Anadir ordenacion

Ciudades ya tiene patron:

```
?sort=campo  
?sort=-campo
```

Archivo:

```
src/back/v2/citys-stats.js
```

Buscar:

```
allowedFields
```

Añadir el campo permitido. No dejar ordenación por campos arbitrarios.

Crear una ruta nueva de API

Pasos:

1. Elegir recurso y version.
2. Abrir archivo de API.
3. Poner ruta concreta antes de rutas parametrizadas.
4. Usar metodo correcto (`GET` , `POST` , `PUT` , `DELETE`).
5. Quitar `_id` si devuelve documentos.
6. Crear service si la usa frontend.
7. Pintarla si hace falta.
8. Crear o adaptar tests.

Ejemplo:

```
app.get(`${BASE_API_URL}/count`, (req, res) => {  
  db.count({}, (err, count) => {  
    if (err) return res.sendStatus(500);  
    return res.status(200).json({ count });  
  });  
});
```

Importante:

En Express importa el orden: `/count` debe ir antes de `/:city/:country`.

Crear v3

Pasos:

1. Crear `src/back/v3`.
2. Copiar como base v2 del recurso.
3. Cambiar `BASE_API_URL` a `/api/v3/...`.

4. Implementar diferencias.
5. Importar en `index.js`.
6. Registrar con `app` y la base correcta.
7. Crear tests v3.
8. Actualizar enlaces si procede.

Cambiar una pantalla o ruta frontend

Rutas actuales se crean por carpetas:

```
frontend-group/src/routes/nueva-ruta/+page.svelte
```

Si hay parametros:

```
frontend-group/src/routes/recurso/editar/[id]/+page.svelte
```

No hace falta registrar manualmente en `App.svelte`, porque se detecta con `import.meta.glob`.

Si quieres enlace visible:

```
frontend-group/src/components/Navbar.svelte
```

Cambiar menu

Archivo:

```
frontend-group/src/components/Navbar.svelte
```

Los enlaces actuales son rutas directas:

```
/
/citys-stats
/analytics
/integrations
```

Cambiar portada

Archivo:

```
frontend-group/src/routes/+page.svelte
```

Cuidado:

- Playwright LCC revisa `data-testid` de la tarjeta LCC.
- No cambiar `member-citys-stats`, `frontend-citys-stats`, `api-v1-citys-stats`, etc. sin actualizar tests.

Cambiar grafica Highcharts

Archivos:

```
frontend-group/src/routes/analytics/+page.svelte  
frontend-group/src/routes/analytics/citys-stats/+page.svelte  
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte  
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Patron:

1. Cargar datos.
2. Transformar datos.
3. Cargar Highcharts.
4. Renderizar.
5. Destruir grafica en `onDestroy`.

Añadir ciudad al mapa

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Objeto:

```
coordinates
```

Clave:

```
city|country
```

Ejemplo:

```
"madrid|spain": { lat: 40.4168, lon: -3.7038 }
```

20. Errores comunes y soluciones

`npm.ps1` no se puede cargar

Causa:

PowerShell bloquea scripts.

Solucion:

```
npm.cmd install  
npm.cmd run build
```

```
npm.cmd start
```

npm start falla

Comprobar:

- Estas en la raiz del proyecto.
- Ejecutaste `npm install`.
- El puerto `10000` no esta ocupado.
- No hay error de sintaxis en el ultimo cambio.

La web se ve antigua

Causa:

`npm start` sirve `public`, no el codigo fuente de `frontend-group/src`.

Solucion:

```
npm.cmd run build  
npm.cmd start
```

Puerto ocupado

Soluciones:

- Cerrar proceso que usa `10000`.
- Cambiar `PORT`.
- Cambiar temporalmente el valor por defecto en `index.js`.

loadInitialData dice que ya hay datos

`loadInitialData` solo inserta datos si la coleccion esta vacia. Si ya existen registros, en LCC devuelve los datos actuales o informa de que no se insertan duplicados.

Solucion rapida para reiniciar ciudades:

```
Invoke-RestMethod -Method Delete http://localhost:10000/api/v2/citys-stats  
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats/loadInitialData
```

API devuelve 400

Puede ser:

- Body incompleto.
- Campo extra.
- Numero no valido.
- Query invalida.

- URL y body no coinciden en `PUT`.

Mirar:

- `normalize...`
- `hasExact...`
- `buildSearchQuery`

API devuelve `409`

Significa conflicto.

Ejemplos:

- Ciudad duplicada por `city + country`.

API devuelve `404`

Significa recurso no encontrado.

Comprobar:

- Identificador correcto.
- Texto normalizado.
- Recurso no borrado previamente.

API devuelve `405`

Significa metodo no permitido.

Ejemplos:

```
POST /api/v2/citys-stats/tokyo/japan
PUT /api/v2/citys-stats
```

Frase:

No es un fallo, es parte del contrato REST.

Integraciones fallan

Puede pasar por:

- API externa dormida.
- Timeout.
- Error en formato JSON externo.
- Problema temporal de red.

Mirar:

- `fetchJson`
- `safeExternal`
- `integrationErrors`

Frase:

Si una fuente externa falla, el backend conserva el resto y comunica el fallo como error parcial.

21. Decisiones tecnicas

Express como servidor unico

Decision:

Usar un unico servidor Express para API y frontend compilado.

Ventaja:

- Facil despliegue en Render.
- Misma origen para frontend y API en produccion.
- Fallback sencillo para rutas SPA.

NeDB como persistencia

Decision:

Usar NeDB con un archivo `.db` por recurso.

Ventaja:

- Sencillo para la practica.
- No requiere servidor de base de datos externo.
- Documentos JSON faciles de entender.

Limitacion:

- No es una base preparada para alta concurrencia o produccion real a gran escala.

Versionado v1/v2

Decision:

Mantener v1 y v2 conviviendo.

Ventaja:

- No se rompe el contrato anterior.
- Se pueden anadir mejoras.

- El frontend puede elegir la version que necesita.

Validacion estricta

Decision:

Rechazar campos extra o cuerpos incompletos.

Ventaja:

- Contrato claro.
- Tests mas predecibles.
- Evita datos basura.

Proxy propio para integraciones

Decision:

El navegador no llama directamente a APIs externas en LCC; llama a nuestro backend.

Ventaja:

- Evita problemas de Same Origin Policy.
- Centraliza timeout y errores.
- Normaliza formatos antes de pintar.

Rutas por carpetas en frontend

Decision:

Usar `+page.svelte` y `import.meta.glob` para descubrir pantallas.

Ventaja:

- Anadir ruta suele ser crear carpeta y archivo.
- Las rutas directas funcionan con fallback Express.

22. Limitaciones y mejoras futuras

Limitaciones actuales:

- NeDB es local y sencillo, no una base empresarial.
- Las integraciones externas dependen de servicios de terceros y Render puede estar dormido.
- Algunas APIs externas pueden cambiar formato.
- Algunas partes de otros recursos tienen estilos y tests menos homogeneos que LCC.
- No hay autenticacion de usuarios.
- No hay migraciones de esquema para cambios grandes.

Mejoras futuras:

- Migrar persistencia a PostgreSQL, MongoDB o similar.
- Anadir tests e2e homogéneos para todos los recursos.
- Anadir cache persistente para integraciones externas.
- Anadir observabilidad: logs estructurados y métricas.
- Mejorar validación con un schema formal como JSON Schema o Zod.
- Crear una API v3 si se necesitan cambios incompatibles.
- Unificar criterios visuales entre todas las pantallas.

Comprobaciones externas no verificables desde el código:

- Videos personales, si la entrega los exige.
- Informes Togg/efforts, si la entrega los exige.
- PR formal asociado a issue done y milestone/release, si la entrega lo pide.

Estos puntos no se pueden confirmar mirando solo el repositorio local, así que no deben presentarse como errores del proyecto ni como funcionalidades pendientes del código.

23. Guion de defensa oral

Esta sección está pensada para estudiar y también para usarla como guion real el día de la defensa. La idea no es leer palabra por palabra como si fuera un texto memorizado, sino entender el orden, repetir las frases clave y saber qué pantalla o archivo abrir en cada momento.

Regla de oro:

```
problema -> arquitectura -> API -> frontend -> funciones -> integraciones -> pruebas -> cierre
```

Si te pierdes durante la defensa, vuelve a esta frase:

Mi proyecto no es solo una tabla. Es una cadena completa: el usuario actúa en Svelte, el frontend llama con `fetch`, Express valida la petición, NeDB guarda o consulta datos, y la respuesta se transforma en CRUD, gráficas, mapa e integraciones.

Preparación antes de entrar

Tener abiertas estas pestañas en el navegador:

1. <https://sos2526-29.onrender.com/>
2. <https://sos2526-29.onrender.com/citys-stats>
3. <https://sos2526-29.onrender.com/analytics/citys-stats>
4. <https://sos2526-29.onrender.com/analytics/citys-stats/map>
5. <https://sos2526-29.onrender.com/integrations/citys-stats>
6. <https://sos2526-29.onrender.com/analytics>

Tener abiertos estos archivos en el editor:

1. `index.js`
2. `src/back/v2/citys-stats.js`
3. `src/back/v1/citys-stats.js`
4. `frontend-group/src/services/citysStatsApi.js`
5. `frontend-group/src/routes/citys-stats/+page.svelte`
6. `frontend-group/src/routes/analytics/citys-stats/+page.svelte`
7. `frontend-group/src/routes/analytics/citys-stats/map/+page.svelte`
8. `frontend-group/src/routes/integrations/citys-stats/+page.svelte`
9. `frontend-group/src/routes/analytics/+page.svelte`

Comandos preparados por si piden ejecutar en local:

```
npm.cmd install
npm.cmd --prefix frontend-group install
npm.cmd run build
npm.cmd start
```

Pruebas preparadas por si piden validar:

```
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Guion de 30 segundos

Usalo si el profesor pide una explicacion muy breve.

Nuestro proyecto es una aplicacion web con backend Express, base de datos NeDB y frontend Svelte. El grupo trabaja con tres recursos: vinos, desastres naturales y estadisticas de ciudades. Mi parte es `citys-stats`, una API REST para gestionar ciudades, paises y poblacion estimada en 2025. He implementado CRUD, validacion estricta, busqueda, filtros, ordenacion, paginacion, analytics, mapa e integraciones externas. La defensa importante es que el dato recorre todo el sistema: se crea o consulta en la interfaz, pasa por un service con `fetch`, llega a Express, se valida, se guarda o lee en NeDB y vuelve al frontend convertido en tabla, grafico, mapa o widget.

Guion de 90 segundos

Usalo cuando haya poco tiempo, pero quieras cubrir todo lo importante.

1. Abrir `index.js`.
2. Decir:

Aqui arranca el servidor. Se crea la aplicacion Express, se activan middlewares como `cors` y `express.json`, se inicializan las bases NeDB de cada recurso, se registran las APIs y se sirve el

frontend compilado desde `public` .

1. Abrir `src/back/v2/citys-stats.js` .
2. Decir:

Este es el contrato REST principal de mi recurso. Aqui estan `GET` , `POST` , `PUT` y `DELETE` . Tambien estan la busqueda libre con `q` , los filtros exactos, la ordenacion, la paginacion y la validacion estricta para que no entren campos incorrectos.

1. Abrir `frontend-group/src/services/citysStatsApi.js` .
2. Decir:

Este service separa la interfaz de la API. Las pantallas no construyen todas las peticiones a mano, sino que llaman a funciones como `getAllCitysStats` , `createCityStat` , `updateCityStat` o `deleteCityStat` .

1. Abrir `frontend-group/src/routes/citys-stats/+page.svelte` .
2. Decir:

Esta es la pantalla CRUD. Aqui se cargan datos, se busca, se crea, se edita y se borra. Los eventos del usuario acaban llamando al service y despues se refresca la lista.

1. Abrir `src/back/v1/citys-stats.js` .
2. Decir:

Aqui estan las integraciones externas. He decidido hacerlas desde el backend para no depender directamente del navegador, controlar errores y normalizar respuestas distintas.

1. Abrir `frontend-group/src/routes/integrations/citys-stats/+page.svelte` .
2. Decir:

Esta vista consume los endpoints proxy y transforma los datos externos en widgets y graficas. Si una API externa falla, la pantalla puede seguir mostrando el resto.

1. Abrir `frontend-group/src/routes/analytics/+page.svelte` .
2. Decir:

Aqui esta el trabajo grupal: un widget comun que combina los tres recursos sin mezclar unidades directamente, usando metricas normalizadas.

Guion principal de 8 a 10 minutos

Este es el guion recomendado si la defensa permite explicar con calma.

Minuto 0: apertura

Pantalla:

```
/
```

Frase:

Buenos dias. Somos el grupo SOS2526-29. La aplicacion permite consultar y visualizar datos de tres recursos: `wine-stats`, `natural-disasters` y `citys-stats`. Mi parte es `citys-stats`, que trabaja con ciudades, paises y poblacion estimada para 2025. Voy a explicar mi recurso de punta a punta: API, base de datos, frontend, graficas, mapa, integraciones y pruebas.

Idea que debe quedar clara:

No presento solo una pagina visual. Presento una aplicacion completa con backend, persistencia, contrato REST y frontend conectado.

Minuto 1: arquitectura general

Archivo:

```
index.js
```

Frase:

El servidor se centraliza en `index.js`. Aqui Express recibe peticiones HTTP, interpreta JSON con `express.json`, permite llamadas desde el frontend con CORS, inicializa las bases NeDB y registra las rutas de cada recurso. Despues, para produccion, sirve el frontend compilado desde `public`.

Explicacion sencilla:

Express es la entrada del backend. NeDB guarda los datos. Svelte es la parte que ve el usuario. El navegador no toca la base de datos directamente; siempre pasa por la API.

Si preguntan por el flujo:

```
Usuario -> Svelte -> service fetch -> Express -> NeDB -> Express -> service -> Svelte
```

Frase para defender:

Esta separacion es importante porque cada capa tiene una responsabilidad concreta. La pantalla no valida como servidor, el servidor no dibuja graficas y la base de datos no sabe nada de botones.

Minuto 2: API REST principal

Archivo:

```
src/back/v2/citys-stats.js
```

Pantalla o URL:

```
/api/v2/citys-stats
```

Frase:

La API principal de mi recurso esta en `/api/v2/citys-stats`. Uso REST porque el recurso es `citys-stats` y las operaciones se expresan con metodos HTTP: `GET` para leer, `POST` para crear, `PUT` para actualizar y `DELETE` para borrar.

Explicacion para alguien sin base:

Una API REST es como un mostrador ordenado. La URL dice sobre que datos hablamos y el metodo dice que queremos hacer con esos datos.

Ejemplos que se pueden decir:

```
GET /api/v2/citys-stats
POST /api/v2/citys-stats
PUT /api/v2/citys-stats/madrid/spain
DELETE /api/v2/citys-stats/madrid/spain
```

Frase clave:

En mi caso la clave del recurso no es un `id`, sino la combinacion `city + country`, porque una ciudad siempre se entiende dentro de un pais y ese par identifica el registro.

Minuto 3: validacion y reglas de datos

Archivo:

```
src/back/v2/citys-stats.js
```

Funciones que conviene senalar:

```
hasExactCityFields  
normalizeCityStat
```

Frase:

Antes de guardar datos, la API comprueba que el objeto tenga exactamente los campos esperados y que los valores sean coherentes. Esto evita que entren registros con campos de mas, campos ausentes o tipos incorrectos.

Explicacion sencilla:

La validacion es como una puerta de entrada. Si el dato no tiene la forma correcta, no pasa a la base de datos.

Ejemplo de defensa:

Si envio un campo extra, la API responde `400`. Si intento crear una ciudad que ya existe para el mismo pais, responde `409`. Si intento actualizar una ciudad inexistente, responde `404`.

Frase para funciones:

`hasExactCityFields` comprueba la estructura. `normalizeCityStat` limpia y transforma datos para que la base guarde una version uniforme.

Minuto 4: CRUD en frontend

Pantalla:

```
/citys-stats
```

Archivos:

```
frontend-group/src/services/citysStatsApi.js  
frontend-group/src/routes/citys-stats/+page.svelte
```

Frase:

La pantalla CRUD no llama directamente a rutas escritas por todas partes. Para eso existe el service `citysStatsApi.js`, que agrupa las llamadas `fetch`. La pantalla Svelte se centra en

estado, formularios, eventos y renderizado.

Demo recomendada:

1. Pulsar cargar datos iniciales si la base esta vacia.
2. Mostrar la tabla.
3. Buscar un pais o ciudad.
4. Ordenar por poblacion descendente.
5. Crear un registro sencillo.
6. Editarlo.
7. Borrarlo.

Frase durante la demo:

Al crear, editar o borrar no modifico la tabla a mano como si fuera una maqueta. Llamo a la API real y despues refresco la lista con los datos que devuelve el backend.

Si preguntan por funciones:

`refreshList` obtiene los datos actuales; `buildSearchQuery` prepara los parametros de busqueda; `validateCityStatForm` evita enviar formularios claramente incorrectos; `handleCreate` envia el POST; `openEdit` carga un registro en el formulario de edicion; `handleUpdate` envia el PUT.

Minuto 5: busqueda, filtros, ordenacion y paginacion

Pantalla:

```
/citys-stats
```

URLs de ejemplo:

```
/api/v2/citys-stats?q=india  
/api/v2/citys-stats?country=china  
/api/v2/citys-stats?sort=-un_2025_population&limit=5
```

Frase:

La v2 no se queda en listar todo. Permite consultar mejor: busqueda libre con `q`, filtros exactos por campo, ordenacion con `sort` y paginacion con `limit` y `offset`.

Explicacion sencilla:

Esto es lo que convierte una API basica en una API util. Si los datos crecen, no quieres traer todo siempre; quieres filtrar, ordenar y pedir solo una parte.

Frase de defensa:

La busqueda y los filtros se expresan como query params porque no cambian el recurso, solo cambian la vista de los datos que estoy pidiendo.

Minuto 6: analytics individual

Pantalla:

```
/analytics/citys-stats
```

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
```

Frase:

Esta pantalla cumple la visualizacion individual del recurso. Toma los datos reales de `citys-stats`, los ordena por poblacion y los representa con Highcharts en un grafico circular.

Explicacion sencilla:

La grafica no inventa datos. Solo cambia la forma de mirarlos. La fuente sigue siendo la API.

Funciones importantes:

```
loadAnalytics  
loadHighcharts  
renderChart
```

Frase para funciones:

`loadAnalytics` carga los datos desde la API; `loadHighcharts` importa la libreria cuando hace falta; `renderChart` prepara la serie, destruye la grafica anterior y crea la nueva.

Minuto 7: mapa

Pantalla:

```
/analytics/citys-stats/map
```

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Frase:

El mapa anade una lectura geografica. Cada ciudad se coloca con coordenadas y se representa con una burbuja cuyo tamano o color depende de la poblacion.

Explicacion sencilla:

Una tabla te dice los numeros, pero un mapa te ayuda a entender distribucion y concentracion geografica.

Funciones importantes:

```
keyFor  
colorFor  
radiusFor  
renderMap
```

Frase para funciones:

`keyFor` identifica cada ciudad, `colorFor` decide el color, `radiusFor` calcula el tamano y `renderMap` pinta o actualiza el mapa.

Minuto 8: integraciones externas

Pantalla:

```
/integrations/citys-stats
```

Archivos:

```
src/back/v1/citys-stats.js  
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Frase:

Las integraciones externas estan pensadas como proxy propio. El frontend no llama directamente a todas las APIs externas. Llama a mi backend y mi backend consulta, normaliza y devuelve una respuesta controlada.

Explicacion sencilla:

Cada API externa responde de una forma distinta. El proxy sirve para traducir todas esas respuestas a una forma mas comoda para mi frontend.

Funciones backend importantes:

```
fetchJson  
safeExternal  
buildIntegratedCityBase  
buildIntegratedCity  
buildCityCountrySummaries
```

Funciones frontend importantes:

```
safeLoad  
loadIntegrations  
renderIntegrationCharts  
destroyIntegrationCharts
```

Frase de defensa:

La decision de integrar por pais y no por ciudad se debe a que muchas APIs externas trabajan mejor con codigos de pais, ISO3 o nombres de pais. Por eso primero agrego mis ciudades por pais y despues cruzo con fuentes externas.

Minuto 9: analytics grupal

Pantalla:

```
/analytics
```

Archivo:

```
frontend-group/src/routes/analytics/+page.svelte
```

Frase:

La vista grupal combina los tres recursos en un unico widget. Como cada recurso mide cosas distintas, no mezclamos unidades directamente; usamos una comparacion normalizada para que la grafica sea defendible.

Explicacion sencilla:

No tiene sentido sumar poblacion, numero de desastres y datos de vino como si fueran la misma unidad. Por eso se normaliza y se muestra una comparacion relativa.

Frase de defensa:

Aquí se ve el trabajo de grupo: tres APIs independientes conectadas en una visualización común.

Minuto 10: pruebas, calidad y cierre

Archivos o comandos:

```
tests/LCC/pruebas-lcc-v2.json
tests/LCC/playwright.config.js
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Frase:

Para validar que no es solo visual, hay pruebas de API con Newman/Postman y pruebas e2e con Playwright. Las primeras verifican endpoints, códigos HTTP y respuestas. Las segundas simulan acciones reales en navegador.

Cierre:

En resumen, mi parte cubre el ciclo completo: modelo de datos, API REST versionada, validación, persistencia, CRUD, búsqueda, filtros, ordenación, paginación, analytics, mapa, integraciones externas, proxy propio y pruebas. Lo importante es que cada pantalla se puede explicar desde el código y cada función tiene una responsabilidad concreta.

Demo exacta recomendada

Orden ideal para enseñar en directo:

1. Abrir `/`.
2. Entrar en `/citys-stats`.
3. Pulsar `Load initial data` si hace falta.
4. Buscar `india` o `china`.
5. Ordenar por población descendente.
6. Crear una ciudad de prueba.
7. Editar la ciudad creada.
8. Borrar la ciudad creada.
9. Abrir `/api/v2/citys-stats?sort=-un_2025_population&limit=5`.
10. Abrir `/analytics/citys-stats`.
11. Abrir `/analytics/citys-stats/map`.
12. Abrir `/integrations/citys-stats`.
13. Abrir `/analytics`.

14. Volver al código y explicar una función concreta si lo piden.

Frase para acompañar la demo:

Voy a hacer una operación real. Si creo un registro, pasa por el formulario, el service, la API, la validación y la base de datos. Después la tabla se refresca pidiendo de nuevo al backend.

Guion para explicar una función cualquiera

Cuando el profesor señale una función, usa siempre este orden:

1. Decir en qué archivo está.
2. Decir quién la llama.
3. Decir qué recibe.
4. Decir qué comprueba.
5. Decir qué transforma.
6. Decir qué devuelve.
7. Decir qué error evita.

Plantilla:

Esta función está en `[archivo]`. La llama `[otra función o pantalla]` cuando `[situación]`. Recibe `[parámetros]`. Primero valida o prepara `[dato]`, después transforma `[dato]` y finalmente devuelve `[resultado]`. Es importante porque evita `[error]` y mantiene separada la responsabilidad de `[capa]`.

Ejemplo con `normalizeCityStat`:

`normalizeCityStat` está en el backend de `citys-stats`. Se usa antes de guardar o devolver datos. Recibe un objeto de ciudad, limpia campos de texto y convierte la población a número. Es importante porque evita guardar datos con formatos inconsistentes.

Ejemplo con `refreshList`:

`refreshList` está en la pantalla CRUD. La llama la pantalla al cargar, buscar, crear, editar o borrar. Prepara la consulta, llama al service y actualiza el estado visible. Es importante porque centraliza la recarga de datos y evita repetir la misma llamada en muchos sitios.

Ejemplo con `safeExternal`:

`safeExternal` está en el backend de integraciones. Envuelve llamadas a APIs externas para que un fallo externo no rompa toda la respuesta. Es importante porque en integraciones reales no

controlamos la disponibilidad de terceros.

Frases perfectas para momentos difíciles

Si preguntan "por que v1 y v2":

Porque versionar permite evolucionar la API sin romper clientes anteriores. En mi caso, v2 concentra el CRUD mejorado y v1 mantiene endpoints anteriores e integraciones.

Si preguntan "por que el frontend llama a v1 en integraciones":

Porque esas rutas concretas de integracion estan implementadas bajo `/api/v1/citys-stats/integrations`. La v2 se usa como contrato CRUD principal.

Si preguntan "por que `citys` y no `cities`":

Porque `citys-stats` ya es el contrato publico del proyecto. Aunque linguisticamente `cities` seria mejor, cambiar la ruta ahora romperia tests, enlaces y documentacion.

Si preguntan "por que NeDB":

Porque encaja con una practica academica: persistencia local, documentos JSON, instalacion sencilla y sin depender de un servidor de base de datos externo.

Si preguntan "por que no se llama directamente a APIs externas desde Svelte":

Porque el backend puede controlar errores, evitar problemas de CORS, ocultar detalles de integracion y normalizar respuestas antes de que lleguen al frontend.

Si preguntan "que pasa si algo falla":

El proyecto intenta fallar de forma controlada. En API se devuelven codigos HTTP coherentes. En integraciones se capturan fallos externos. En frontend se muestran mensajes y se evita que toda la pantalla quede rota.

Plan B si falla algo en directo

Si Render tarda en despertar:

Render puede dormir la aplicacion cuando lleva tiempo sin uso. Mientras arranca, puedo explicar el codigo y despues volver a la pestana.

Si falla una API externa:

Esta es precisamente una razon para usar proxy y gestion de errores. Las integraciones dependen de servicios externos; el proyecto captura el fallo y puede seguir mostrando el resto.

Si la base ya tiene datos y `loadInitialData` no carga:

El endpoint evita duplicar datos iniciales. Si la base no esta vacia, responde indicando que ya hay datos.

Si una demo de crear ciudad da conflicto:

Eso significa que ya existe el par `city + country` . La API devuelve `409` , que es el codigo correcto para conflicto de recurso.

Si preguntan algo que no recuerdas:

Lo miraria desde el flujo. Primero localizo la pantalla, despues el service que llama al backend y por ultimo el endpoint de Express que resuelve la peticion.

Cierre recomendable

La idea importante es que no hay una pantalla aislada: hay una cadena completa. Svelte recoge una accion, el service hace `fetch` , Express valida, NeDB guarda o consulta, y el frontend transforma la respuesta en tabla, formulario, grafica, mapa o integracion. Por eso el proyecto se puede defender desde usuario, desde API y desde codigo.

24. Preguntas dificiles y respuestas

Por que REST y no rutas tipo `/crearCiudad`

Porque REST identifica recursos con URLs y usa metodos HTTP para las acciones. La URL dice "que recurso" y el metodo dice "que accion".

Por que `POST` crea en la coleccion

Porque antes de crear no existe la URL del recurso concreto. La coleccion es donde nace el nuevo recurso.

Por que `PUT` lleva identificador en URL

Porque actualiza un recurso existente. Ademas se comprueba que URL y body coincidan para evitar modificar otro registro por error.

Por que se usan query params

Porque filtros, busqueda, orden y paginacion no cambian el recurso base; solo cambian la vista que se devuelve.

Por que NeDB

Porque para esta practica necesitamos persistencia sencilla en archivos. NeDB permite documentos JSON sin instalar una base externa. Cada recurso tiene su propio `.db`.

Por que se quita `_id`

Porque `_id` es interno de NeDB. La API publica no debe depender de detalles de almacenamiento.

Por que hay v1 y v2

Porque v1 mantiene compatibilidad y v2 anade mejoras sin romper clientes previos. En LCC, v2 se usa para CRUD y v1 aloja integraciones.

Por que algunas rutas usan v1 en frontend

Porque `citysStatsIntegrations.js` llama a integraciones que estan implementadas en `/api/v1/citys-stats/integrations/...`.

Por que `citys-stats` esta escrito asi

Porque es el contrato publico usado por rutas, tests y documentacion. Cambiarlo romperia enlaces y entregables.

Por que claves compuestas

Porque no todos los datasets necesitan un `id` numerico. En LCC, una ciudad se identifica de forma natural con `city + country`.

Que pasa si mando un campo de mas

La API responde `400`, porque la validacion exige estructura exacta.

Por que `DELETE` devuelve a veces `204`

Porque el borrado fue correcto y no hace falta devolver cuerpo.

Que hace CORS

Permite que el frontend en desarrollo, que puede estar en otro puerto, llame al backend sin bloqueo del navegador.

Como se despliega el frontend

Vite compila `frontend-group/src` hacia `public`. Express sirve `public` con `express.static`.

Por que funcionan rutas directas como `/analytics`

Porque Express devuelve `index.html` para rutas que no son `/api`, y Svelte resuelve la pantalla por `window.location.pathname`.

Donde esta el proxy propio

En `src/back/v1/citys-stats.js`, bajo `/api/v1/citys-stats/integrations/...`.

Que pasa si una API externa falla

`fetchJson` y `safeExternal` controlan el error. La vista puede mostrar datos parciales e informar avisos.

Por que no se muestra JSON crudo

Porque JSON es el formato de intercambio. En la interfaz se transforma en widgets, tablas, tarjetas y metricas.

Como se relacionan datos locales y externos

Primero se agregan ciudades por pais. Despues se normalizan nombres de pais e ISO3 para cruzarlos con las fuentes externas.

Donde se ve el trabajo de grupo

En los tres recursos, la portada, las APIs, tests, `/analytics` y `/integrations`.

25. Glosario para personas no tecnicas

Termino	Explicacion
Backend	Parte del sistema que recibe peticiones, valida datos y habla con la base
Frontend	Parte visible que usa el usuario en el navegador
API	Forma ordenada de pedir o enviar datos a un servidor
REST	Estilo de API basado en recursos, URLs y metodos HTTP
Recurso	Tipo de dato gestionado, por ejemplo <code>citys-stats</code>
CRUD	Crear, leer, actualizar y borrar
JSON	Formato de texto para intercambiar datos

Termino	Explicacion
NeDB	Base de datos local que guarda documentos en archivos
Endpoint	Ruta concreta de una API
Query param	Parametro de URL como <code>?limit=5</code>
Proxy	Servidor intermedio que llama a otra API por nosotros
SPA	Aplicacion web que carga una vez y cambia pantallas con JavaScript
Build	Version compilada y lista para servir
Test e2e	Prueba que simula acciones reales en navegador
Newman	Herramienta para ejecutar colecciones Postman
Highcharts	Libreria para graficas y mapas

26. Checklists finales

Antes de defender

```
npm.cmd install
npm.cmd --prefix frontend-group install
npm.cmd run build
npm.cmd start
```

Abrir:

- `http://localhost:10000/`
- `http://localhost:10000/citys-stats`
- `http://localhost:10000/analytics`
- `http://localhost:10000/analytics/citys-stats`
- `http://localhost:10000/analytics/citys-stats/map`
- `http://localhost:10000/integrations`
- `http://localhost:10000/integrations/citys-stats`
- `http://localhost:10000/api/v2/citys-stats`
- `http://localhost:10000/api/v1/citys-stats/integrations/summary?limit=8`

Tests utiles

```
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Despues de tocar backend

- Reiniciar `npm start`.
- Probar endpoint directo.

- Ejecutar Newman del recurso.

Despues de tocar frontend

- Ejecutar `npm.cmd run build` si se va a probar con `npm start`.
- Abrir ruta afectada.
- Revisar consola del navegador.
- Ejecutar Playwright si afecta a flujo visible.

Despues de tocar campos

- Ejecutar `rg "nombre_del_campo"`.
- Actualizar backend, frontend, analytics, integraciones y tests.
- Probar crear y editar.

Checklist LCC D03

- CRUD v2 accesible.
- Documentacion Postman v2 enlazada.
- Carga inicial disponible.
- Búsqueda libre `q`.
- Filtros exactos.
- Ordenacion `sort`.
- Paginacion `limit` y `offset`.
- Grafico individual no `line`.
- Mapa geoespacial.
- Integraciones por pais.
- 7 APIs integradas.
- 3 APIs no SOS.
- 4 APIs SOS externas.
- Proxy propio.
- Vista `/integrations`.
- Widget grupal unico en `/analytics`.

27. Anexos tecnicos

Comandos API rapidos para `citys-stats`

```
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats/loadInitialData
Invoke-RestMethod "http://localhost:10000/api/v2/citys-stats?q=india"
Invoke-RestMethod "http://localhost:10000/api/v2/citys-stats?sort=-un_2025_population&limit=5"
```

Crear ciudad:

```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:10000/api/v2/citys-stats `
  -ContentType "application/json" `
  -Body '{"city":"madrid","country":"spain","un_2025_population":7000000}'
```

Borrar todo:

```
Invoke-RestMethod -Method Delete http://localhost:10000/api/v2/citys-stats
```

Rutas de documentacion Postman

```
/api/v1/citys-stats/docs
/api/v2/citys-stats/docs
```

Regla de oro para cambios

Si cambia una API:

```
backend -> service -> pantalla -> tests -> build
```

Si cambia solo una grafica:

```
pantalla analytics/integrations -> renderChart/renderMap -> probar ruta
```

Si cambia un campo:

```
backend + initialData + frontend + analytics + tests
```

Estado final de docs

Los documentos antiguos de defensa dentro de docs/ ya no deben usarse para estudiar, preparar cambios ni defender el proyecto. Se eliminan como fuentes independientes para evitar duplicar informacion y para que nadie tenga que buscar explicaciones repartidas en varios sitios.

Las unicas referencias activas dentro de docs/ deben ser:

```
docs/DEFENSA_COMPLETA.md
```

El PDF antiguo puede usarse solo como copia derivada si se regenera desde este Markdown.